

Warsaw University of Technology

FACULTY OF ELECTRICAL ENGINEERING



Department of Information and Measurement Systems

Master's diploma thesis

in the field of study Applied Computer Sciences

and specialisation Computer Sciences in Business

Analysis and Evaluation of CI/CD Platforms Integrated with Git Hosting Services: A Case
Study of GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines

inż. Krzysztof Żerebecki

student record book number 336428

supervisor

dr hab. inż. Andrzej Majkowski

WARSZAWA 2026

Analysis and Evaluation of CI/CD Platforms Integrated with Git Hosting Services: A Case Study of GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines

Abstract

The aim of this master's thesis is to conduct a comparative analysis of the CI/CD pipelines offered by the three most popular Git hosting version control system providers. This thesis provides a holistic comparison of the GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines services. The first chapter defines the objectives, scope, research questions and motivations behind selecting this research topic. The theoretical part covers the foundations of the research area. It covers version control systems, with emphasis on the Git system, which is followed by a chapter on continuous integration and continuous delivery, CI/CD pipelines, and the DevOps methodology. Another important issue is a description of each of the three providers and a comparison of their terminology and the way they define key elements of the working environment. Last subchapter describes formulas from the queueing theory used in the study. The next part of the work describes all the steps necessary to create a repeatable and reliable testing environment. First, it describes in detail how comparable setups are configured for each provider. Next, it defines general CI/CD pipeline structures and briefly describes self-hosted runners configuration on Kubernetes cluster. The last element of this part is a detailed description of all the solutions implemented to collect and process data from CI/CD pipelines, in particular data pipeline configured in AWS Cloud. The results section of the thesis, describes and analyses in verbal, tabular, and graphical form the outcomes collected from many different test cases conducted in the execution environment, created as part of this master's thesis. It is divided in two clear subsections, with the first being a general user-centric evaluation and the second a performance evaluation. Apart from this division, the latter part is divided into pipeline duration and pipeline call latency analysis, which is presented in the context of the queueing theory. In the conclusion of the thesis, final recommendations are formulated regarding the use of each provider's CI/CD solutions, depending on the business needs of the project, based on the research conducted as part of this master's thesis. The recommendations are set in the context of the research limitations and of possible further directions for the development of studies on the performance and use of CI/CD pipelines.

Keywords: CI/CD pipelines, software engineering, IT systems architecture and engineering, version control systems, Git, DevOps, software testing, queueing models in computer systems

Analiza i ocena platform CI/CD zintegrowanych z usługami hostingu Git na przykładzie GitHub Actions, GitLab CI/CD i Bitbucket Pipelines

Streszczenie

Celem niniejszej pracy magisterskiej jest analiza porównawcza potoków wdrożeniowych CI/CD dostępnych u trzech najpopularniejszych dostawców systemów kontroli wersji w ramach hostingu Git. W ramach pracy porównano holistycznie usługi GitHub Actions, GitLab CI/CD i Bitbucket Pipelines. Rozdział pierwszy definiuje cele pracy, zakres badań, pytania badawcze i motywacje przyświecające podjęciu niniejszego tematu. Część teoretyczna omawia niezbędne podstawy obszaru badań. Obejmuje systemy kontroli wersji, ze szczególnym uwzględnieniem systemu Git, po czym następuje rozdział poświęcony ciągłej integracji i ciągłemu dostarczaniu, potokom wdrożeniowym CI/CD oraz metodologii DevOps. Kolejną ważną kwestią jest opis każdego z trzech dostawców oraz porównanie ich terminologii i sposobu definiowania kluczowych elementów środowiska pracy. Ostatni podrozdział opisuje wzory z teorii kolejek wykorzystane w badaniu. W kolejnej części pracy opisano wszystkie kroki niezbędne do stworzenia powtarzalnego środowiska testowego. Najpierw przedstawiono, w jaki sposób skonfigurowano porównywalne ustawienia dla każdego dostawcy. Następnie zdefiniowano ogólne struktury potoków wdrożeniowych CI/CD i krótko opisano konfigurację lokalnie hostowanych maszyn wykonawczych dla potoków w klastrze Kubernetes. Ostatnim elementem tej części jest szczegółowy opis wszystkich rozwiązań wdrożonych w celu gromadzenia i przetwarzania danych z potoków CI/CD, w szczególności potoku danych skonfigurowanego w chmurze AWS. W części poświęconej wynikom pracy opisano i przeanalizowano w formie tekstowej, tabelarycznej i graficznej wyniki zebrane w środowisku wykonawczym, stworzonym w ramach niniejszej pracy magisterskiej. Część ta podzielona jest na dwie wyraźne podsekcje, z których pierwsza stanowi ogólną ocenę zorientowaną na użytkownika, a druga ocenę wydajności. Oprócz tego podziału, druga część jest podzielona na analizę czasu trwania potoku i opóźnienia wywołania potoku, które są przedstawione w kontekście teorii kolejek. W zakończeniu pracy sformułowano ostateczne rekomendacje dotyczące wykorzystania rozwiązań CI/CD każdego dostawcy w zależności od potrzeb biznesowych projektu. Rekomendacje te osadzone są w kontekście ograniczeń oraz dalszych możliwych ścieżek rozwoju badań nad wydajnością i wykorzystaniem potoków CI/CD.

Słowa kluczowe: potoki CI/CD, inżynieria oprogramowania, architektura i inżynieria systemów, systemy kontroli wersji, Git, DevOps, testowanie oprogramowania, modele kolejkowe w systemach komputerowych

Contents

1	Introduction	9
2	Theoretical foundations	11
2.1	Version Control Systems	12
2.2	Contemporary approaches to software integration and delivery	20
2.3	Overview of major Git hosting platforms	24
2.4	Introduction to the queueing theory	36
3	Solution implementation overview	41
3.1	Management repository and provider organisation configurations	42
3.2	Overview of CI/CD pipeline definitions	49
3.3	Self-hosted runner scale set configurations	54
3.4	Test results data pipeline	57
4	CI/CD pipeline evaluation	59
4.1	User-centric evaluation of CI/CD pipelines	60
4.2	Performance evaluation of CI/CD pipelines	66
5	Summary	81
	References	87
	List of Abbreviations and Symbols	91
	List of Figures	93
	List of Tables	95

Chapter 1

Introduction

The introduction chapter presents the context and motivation for this thesis. It explains the role of CI/CD pipelines in modern software development and introduces the three analysed services: GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines. Apart from that, it also outlines the overall structure of the work and briefly describes the theoretical background, the design of the experimental environment, and the analysis of the collected data. In this way, the reader receives a clear overview of how the thesis moves from basic concepts and definitions to controlled measurements and, finally, to the interpretation of results.

The central problem addressed in this thesis is the lack of a systematic and reproducible comparison of CI/CD services integrated with Git hosting platforms. In practice, decisions about which platform to use are often based on individual experience or vendor documentation rather than on quantitative evidence. There is limited data on how these platforms behave under different workloads, how they use managed and self-hosted runners, and how they influence pipeline execution time, queueing behaviour, scalability, and configuration effort. This gap makes it difficult for teams and organisations to select a platform that fits their technical and business requirements.

The main objective of the thesis is therefore to perform a comparative analysis of CI/CD pipelines provided by GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines, based on both qualitative and quantitative metrics. The work aims to construct a common execution environment, measure performance and queueing characteristics, and derive practical guidelines for platform selection. In this context, the thesis investigates the following research questions:

- How the platforms differ in configuration model and ease of use for typical project workflows?
- How pipeline execution time and start latency depend on code base size and runner environment?

- How observed queueing metrics relate to M/G/1, G/G/1 and G/G/c models for different arrival patterns?

Chapter 2 of this thesis broadens the theoretic aspects, which are indispensable for a full understanding of the subject matter and scope of the master's thesis. It begins by outlining a brief history of Version Control Systems and focuses on describing in detail the one solution, that dominates modern IT industry, namely the Git version control system. Later on, this chapter goes through other topics, such as: Continuous Integration, Continuous Development, DevOps methodology. It also briefly discusses theoretical aspects of using GitHub, GitLab and Bitbucket Git hosting providers, introduces queueing theory fundamentals and only the necessary formulas and definitions from this knowledge realm.

Chapter 3 focuses on the solution implementation overview, which discusses testing environment implementation in details, which may be helpful when understanding the results of this thesis. It fully describes issues such as, how repositories and organizations in each Git hosting provider have been configured, in order to create comparable testing environments, how CI/CD pipelines have been created to test capabilities of each provider, how self-hosted runner scale sets have been implemented in Kubernetes cluster and how test results are collected and processed.

Chapter 4 focuses in detail on analysing the results of the tests in terms of many different scenarios, such as small code base, large code base, provider-hosted runners or self-hosted runners. It contains additional chapter that evaluates CI/CD pipeline configurations from the user-centric perspective, for instance by discussing template utilisation or debugging.

The last chapter is a thesis summary, which collects all the important information from each chapter in order to briefly recap on the results of this master's thesis.

Chapter 2

Theoretical foundations

This chapter provides the theoretical background that supports the research objectives defined in the introduction. It defines the concepts and models that are later used to design the experimental environment and to interpret the observed behaviour of CI/CD pipelines.

First, the chapter presents version control systems and focuses on the Git system as the common base for all analysed platforms. Next, it explains continuous integration, continuous delivery, CI/CD pipelines, and the DevOps methodology, with emphasis on how they structure the software development and delivery process. The chapter then describes GitHub, GitLab, and Bitbucket as version control system providers and compares their naming conventions, terminology, and methods of defining key elements of the working environment, such as organisations, pipelines, jobs, and runners. In the following sections, the chapter introduces the basic elements of queueing theory, including the notation and performance measures used later in the thesis, and presents the M/G/1, G/G/1 and G/G/c models as abstractions for CI/CD workloads with different arrival and service patterns.

The material presented in this chapter establishes a common terminology and analytical framework for the rest of the thesis. It is used in the next chapter to design a repeatable test environment on each platform and in the results chapter to analyse performance, queueing behaviour, and security-related aspects of the evaluated CI/CD configurations.

2.1 Version Control Systems

One of the many well-known truths about life is that any action undertaken either by a man or by a machine is prone to error and imperfection. Historically, the need to improve what has already been created has followed humanity through the ages. Taking books as an example, in the early days of the publishing process it was not uncommon to add errata pages to already printed books so as to correct mistakes discovered after printing, without having to reprint the entire book. Later on, books could also be published in editions to address accumulated errors, updates, and revisions, a practice that persists to this day, unlike errata pages. The introduction of computers as we know them today has created an unprecedented opportunity to simplify the process of creating new versions and detecting or correcting errors at an early stage. In particular, this has proven to be of major importance to the computer program development process because of the ever-growing complexity of computer systems and software. The interdependence of the many pieces that make up modern software results in a situation where, if one piece is updated in a more significant manner, many other dependent pieces may stop working as expected and therefore using correct version of the software piece became essential. Before the World Wide Web era changes to the software were not as frequent as today, yet the need to track changes was still important. Therefore, the first known Version Control System (VCS) was created by Marc Rochkind in late 1972, when he was working for Bell Labs. The system was called the Source Code Control System (SCCS), and it was described by the author as *“a software tool designed to help programming projects control changes to source code. It provides facilities for storing, updating, and retrieving all versions of modules, for controlling updating privileges, for identifying load modules by version number, and for recording who made each software change, when and where it was made, and why”* [1]. This system belongs to the group of Version Control Systems, that is called Local Version Control Systems

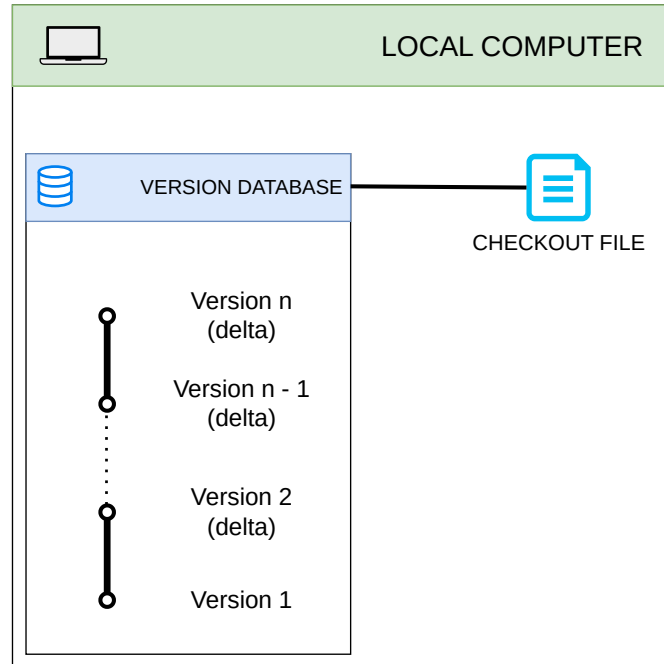


Figure 1. Local version control system scheme

Local version control systems, as the name suggests, are computer applications that store information about new versions of files on a local workstation and this is clearly depicted in the Figure 1. SCCS is a local version control system that allows each file version to be stored efficiently using so-called “deltas”. A version database stores a full copy of the first version of each file and, when a new version modifies that file, stores only information concerning the modified lines. As mentioned by the author in [1], *“All changes are stored in terms of two primitives: insertion of an entire line and deletion of an entire line. This means, for example, that the replacement of a single character on a line is stored as a deletion of the entire old line and an insertion of an entire new line.”* Applying this procedure repeatedly to a file creates what is known as “delta chaining”, which means that reconstructing a selected version requires applying a sequence of deltas to a base version. This can be shown using formula 1 which is placed below.

$$V_0 + \Delta V_1 + \Delta V_2 + \dots + \Delta V_n \rightarrow V_{n+1} \quad (1)$$

This allows to store each software version for various purposes without having to create full multiple copies of each file and thus saving a lot of disk space. A few years later, Revision Control System (RCS) was introduced by Walter F. Tichy as a local tool for maintaining multiple revisions of individual files. This tool implemented many more features than SCCS could offer, including logging,

identification, and merging of revisions. For space efficiency, “RCS stores revisions in the form of deltas, i.e., as differences between revisions” [2], where “the deltas employed by RCS are line-based” [2]. To reduce the cost of retrieving the most common revision, RCS arranges trunk storage so that “The most recent revision on the trunk is stored intact. All other revisions on the trunk are stored as reverse deltas” [2]. This strategy dramatically speeds up retrieval of the most recent version of a file in a project with tens or hundreds of versions, which was problematic for SCCS because it had to reconstruct the most recent version by applying a long chain of deltas to the base file. Currently, local version control systems are mostly overshadowed by a more advanced version control system called Git, which is described in detail later in this chapter. Still, local version control systems might prove useful for projects maintained by a single person, especially when small files change only rarely. For this reason, RCS remains available today, for example as part of common Unix-like environments and distributions. However, this is not the case for the vast majority of IT projects nowadays. It soon became clear that more and more IT projects required large teams of developers to maintain them, and locally managed version control systems had to be regularly synchronised among team members. That is why Centralised Version Control System (CVCS) were created: to enable development teams to collaborate on a project with full synchronisation.

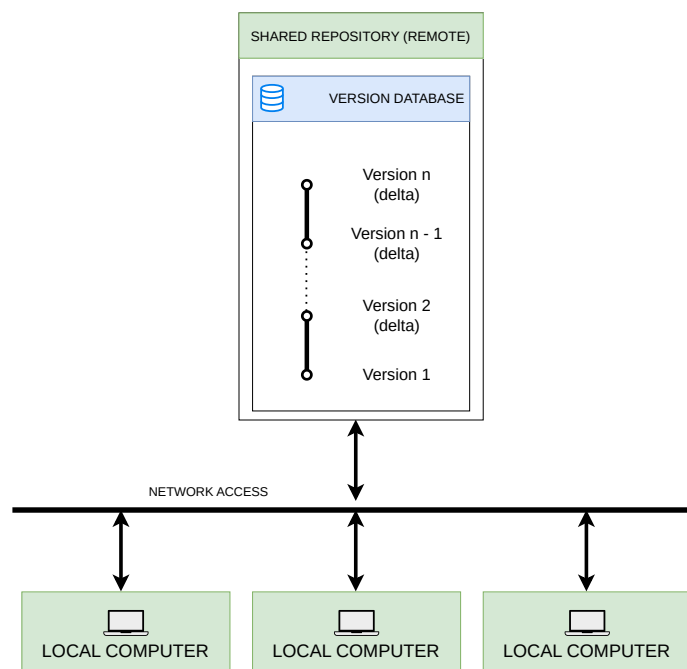


Figure 2. Centralised version control system scheme

Centralised version control systems rely on hosting the version control service and the canonical repository on a single server and accessing it from developers’ workstations over a network (Local Area Networks, Wide Area Networks, and later the Internet, depending on the server’s location). This approach is depicted in the Figure 2. Developers pulled files and directories into a local

workspace, modified them locally, and then synchronised their changes with the central repository (typically by creating updates and, when necessary, merging concurrent changes). Systems like CVS, Subversion, or Perforce were the industry standard for a significant period of time until they were later overshadowed by Git, much like Local version control systems. This architecture offers several advantages: it provides a single “source of truth” for the project history, simplifies user management and access control, enables centralised backups and auditing, and can enforce policies such as file locking when exclusive edits are required. However, it also introduces notable drawbacks: the central server becomes a single point of failure and a performance bottleneck, developers are strongly dependent on network availability and latency, offline work is limited, and concurrent development can lead to frequent update/merge conflicts, especially in large or highly active teams.

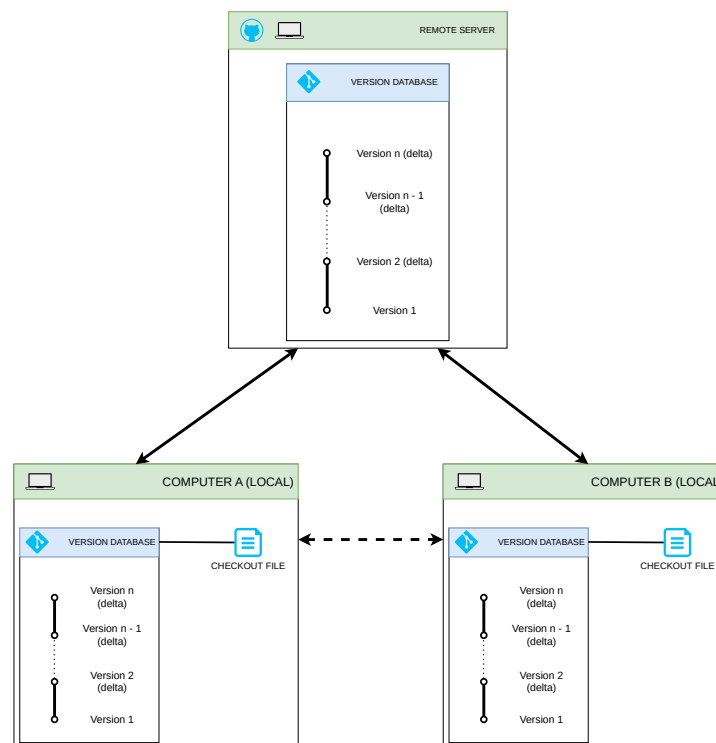


Figure 3. Distributed version control system scheme

Local Version Control Systems allow for creating local project versions without much complexity, but do not allow for extensive collaboration of a development team on a project. Centralised Version Control Systems allow for extensive collaboration of a development team, but make it heavily reliant on a working server and a reliable network connection. Distributed Version Control Systems are a response to these limitations, since they offer the best of both worlds. In distributed version control systems, as shown in Figure 3, each workstation has its own full copy (or clone) of the entire project version database, including the repository history. This allows developers to perform most operations locally – such as committing, branching, inspecting history, and comparing revisions without requiring continuous network access, and to synchronise changes with other repositories only

when needed. Most commonly, distributed version control systems implement remote server for the repository, in order to make it easier for the entire team of the developers to synchronise and manage their versions of the project. In the early 1990s, Sun Microsystems developed TeamWare, an early distributed revision control system in which a workspace contains a complete copy of the project history and there is no notion of a central repository. Larry McVoy was involved in TeamWare's design and later developed BitKeeper, a distributed system released in 2000 and used for Linux kernel development beginning in 2002 [3]. After the free-of-charge availability of BitKeeper for the kernel community was revoked in 2005, Linus Torvalds¹ initiated Git to retain a distributed workflow while meeting following set of requirements [3]: speed, simple design, strong support for non-linear development (thousands of parallel branches), fully distributed, able to handle large projects like the Linux kernel efficiently (speed and data size).

Many historical VCS solutions' architectures emphasize storing efficiently changes made to individual files in the project code base and the version deltas in particular. Git however, is designed to analyse project state based on a form of snapshots [3], which are referred to as "commits" in the Git nomenclature. Git stores all the information regarding project versioning in a hidden '.git' directory on the project root level. When a new version of the project is added to the staging area, Git computes an object identifier from the file contents and stores the corresponding object in its object database if it does not already exist. The last step of the staging process is to update INDEX file, which contains information about the path to the file, its SHA checksum and the file mode, which for instance indicates to Git file permissions. The next step in the process of the Git versioning is "committing". The checksum calculated from each file in the project is now a key for these files inside Git's object database. Values assigned to each file key in this database are file blobs (binary large objects) compressed using zlib/Deflate algorithm [4] from actual changed files. This means that if the file has not been changed in the new revision, Git only points to the already existing blob object from the previous commits, and if the file has been removed entirely from the project, it still can be restored using previous commits. Next, Git tree objects are created. Git represents the project directory structure using tree objects, which mirror the folder and file layout. Each tree node represents each directory in the project and each tree leaf represents each file in the project, by pointing to a blob object created during the staging process or originating from previous commits. Each node SHA ID is calculated using information such as, its children SHA IDs, subdirectory names and its modes. The process is repeated up until it reaches the root of the tree. Commit object creation writes tree objects from the index (bottom-up) and then writes a commit object that references the root tree and parent commit(s). The commit object contains information about the tree root ID, parent (previous) commit ID, author and committer information, timestamps and a commit message, which should be a brief description of what changes are introduced with this particular commit. This provides a combination that allows Git to store information about project versions efficiently, while

¹Linus Torvalds, born on Dec 28th 1969, is a Finnish-American software engineer best known as the creator of the Linux kernel and the Git version control system, which has been developed to be used for Linux kernel development.

allowing developers not to worry, that any file in the project may be lost forever. Typical simple workflow when versioning with Git is shown in the Figure 4.

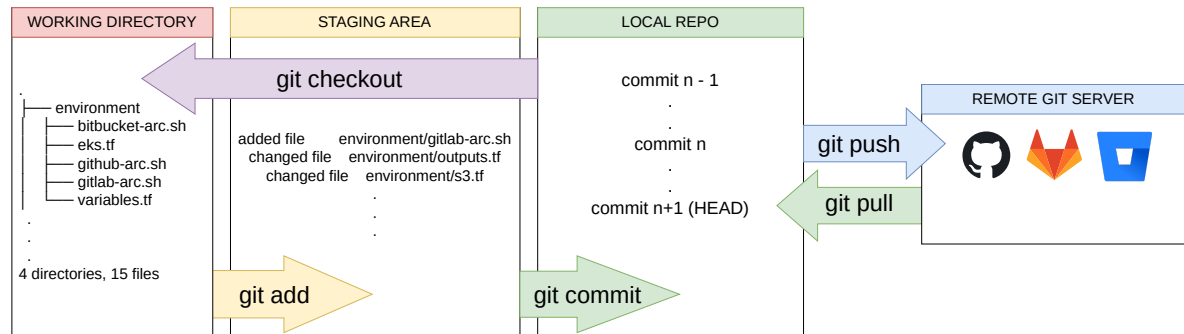


Figure 4. Typical simple versioning scheme with Git

A developer working locally on the full copy of the project repository first adds new changes to the staging area, either using commands like `git add` [5] or options within Integrated Development Environment (IDE). After the changes are added to the staging area, they can be verified once more before moving to the next stage in the process, which is “committing” to the local ‘git’ repository. Committing is done also either using command `git commit` in Command Line Interface (CLI) or using directly IDE. This part is responsible for creating the project’s “snapshot”. However, these changes are introduced locally and if the project is maintained by a team of developers, it is often recommended to “synchronise” your local changes with a remote Git repository. This can be done in two ways, first option is that we can introduce the remote server with our local changes, which is done using `git push` command or IDE. Push transfers local commits/objects to a remote and updates remote. Another option is to “synchronise” our local Git repository by incorporating the changes introduced to the remote repository by other team members, which is done using `git pull` command or our IDE. Pull fetches updates and integrates them into the local repository. Developers can use `git checkout` to switch the currently checked-out branch (or restore selected files) and thereby update the working directory to the chosen repository state. When `git pull` integrates remote changes into the current branch, Git updates the working directory accordingly to reflect the new state. This opportunity for the developers to freely experiment with the code base with minimal risk of losing crucial pieces of the project is one of the many reasons Git has become the dominant solution in the industry.

Another reason why this has happened is that Git allows developers to work simultaneously on different versions of the same project in the same time. This is possible in Git by implementing so called “branches”. A branch in Git terminology is simply a named reference to the most recent commit object in a line of commit history. The reference in Git is just a human-readable pointer to some commit object and references are stored “loosely” under `refs` directory. In particular references under `refs/heads` are all project’s existing branches. Some Git versions also can store this and many

more information in a packed form under packed-refs. However, for the sake of simplifying the explanation of how Git works, the following discussion focuses on the loose format. The default and therefore most common branches in Git are either branch “master” or branch “main”, which is placed under `refs/heads/master` or `refs/heads/main`. Git stores the information about which branch is currently checked out in the working directory using a special file named `HEAD`. This file contains a single line in a following format: `ref: refs/heads/<BRANCH NAME>`. As far as the commit history is concerned, each commit object contains information about parent commit ID and it is also possible that many commit objects link to single parent commit object. Therefore, we can understand the commit history of a Git repository as a directed acyclic graph structure which is shown in the Figure 5.

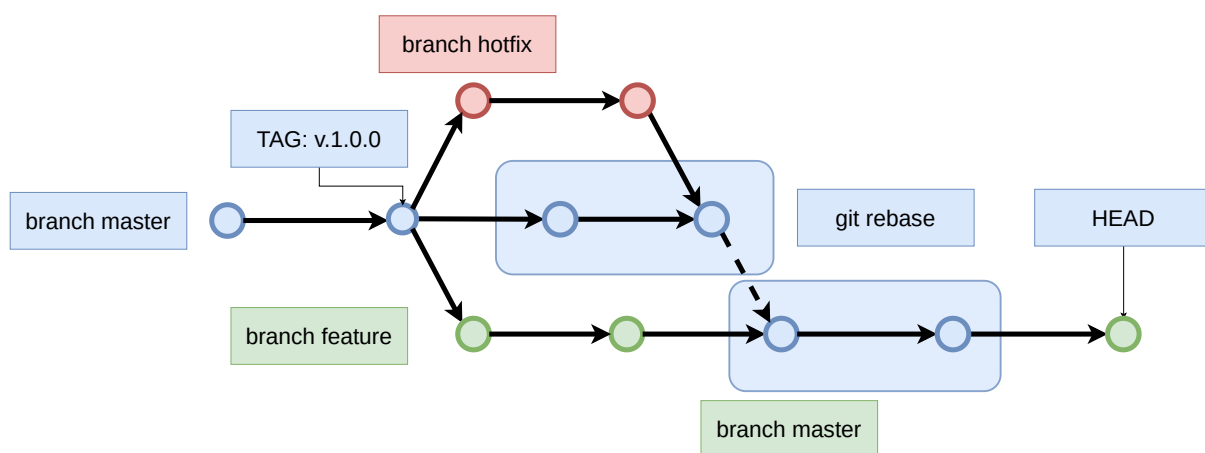


Figure 5. Simplified diagram presenting most common Git branching scenarios

Git branches are mostly used by developers to safely incorporate new changes into the base branch (e.g., master). Sometimes, they can also be used to maintain different adaptations or editions of a repository. For instance, for some IT projects it may be beneficial to keep different codebases depending on the operating system the software should run on. There are a plethora of Git branching strategies, and one of the most popular is the so-called “Gitflow”. As defined in [6], “*Gitflow is an alternative Git branching model that involves the use of feature branches and multiple primary branches*”. This allows teams to separate ongoing development from released code and maintenance work by organising changes into dedicated branches (e.g., feature, release, and hotfix branches). In the Figure 5 new branch “feature” has been checked out from the “master” branch at the v.1.0.0 tag. Git allows for assigning pointers called tags to specific commits in the repository history, so that they can be used for example with the project’s release management. In this repository, the last two commits of the “master” branch have been replayed on top of the “feature” branch using a rebase operation. Git rebase operation simply integrates changes by recreating (replicating) a sequence of commits onto a new base commit and moves the “feature” branch reference to the rewritten history. The original, pre-rebase commits may still be reachable temporarily but are no longer pointed to by the branch reference.

Another branch, named *“hotfix”*, has been created from the same commit as the *“feature”* branch. This is a very common practice that if a team notices a bigger issue with the code being developed on some branch, they create a new branch for fixing the bug and then merge it back into the original branch. Merging in Git is a process of combining changes from one branch into another. There are many possibilities with merging two branches. First of all, if the branch reference on the *“merge-into”* branch points to the same commit that the new branch originates from, Git performs *“fast-forward”* merge and simply moves the pointer of the *“merge-into”* branch forward to the tip commit of the merged branch, because there is no divergent work on it. Second alternative is when there actually is some divergent work on the *“merge-into”* branch. In that case, typically a new merge commit must be created on the *“merge-into”* branch using one of the many strategies Git has to offer. The most common merge strategy is the *“ort”* strategy, in modern Git, *“recursive”* is a synonym for *“ort”*. As defined in [7], *“This is the default merge strategy when pulling or merging one branch. This strategy can only resolve two heads using a 3-way merge algorithm”*. In a *“three-way merge”* algorithm Git is looking at two commits, also called tips of the branches, and compares them with a base commit, which is the *“best”* common ancestor of both branch tips (the merge base) selected by Git. For each branch Git gathers chunks of code that have been changed since the base commit and then tries to merge both changes into one new commit. If the changes overlap incompatibly, a merge conflict may occur and a developer must address conflicts manually. However, sometimes a developer wants to ignore the impact of a specified branch on conflicts in changes. Therefore the *“ort”* strategy supports two major conflict-resolution options. The first option is *“ours”*. This option tells Git that whenever a conflict occurs when merging, it has to favour changes made by the branch that we are currently checked-out on (has HEAD reference on). It is worth noticing here, that Git allows also separate from the *“ort”* merge strategy called *“ours”*, which ignores all changes introduced by the other branch while recording it as a branch merge in history. The other *“ort”* option is *“theirs”*, which is an opposite option that tells Git to favour changes made by the merged-in branch (the one, we are merging from). Understanding how merging works is extremely important nowadays, as the IT projects that are bigger in scale, than just a small side-project, often require multiple improvements over time, which would be very difficult to introduce without the ability to freely work on a development branch and then merging working changes into the production code.

2.2 Contemporary approaches to software integration and delivery

Previous subchapter has covered Git in depth, the arguably most popular version control system used in IT projects around the world. According to the “*Stack Overflow*” 2022 Developer Survey [8], an overwhelming 93.87% of respondents use Git as their primary version control system. It is an indispensable tool when it comes to tracking, coordinating, and reviewing changes to source code across many contributors. However, the tool alone – no matter how flawless it may be – is often not enough for successful collaboration in a larger development team or on enterprise-level projects. There are several potential risks that may occur when using Git without introducing any common development strategy. First of all, if development branches are not merged into main (or another integration branch) for a long period, a lot of merge conflicts may occur while merging and some of them may require further clarification or even in the worst-case scenario a series of team meetings to figure out which changes should be introduced and which existing chunks of code can be removed. This is an obvious risk which may lengthen the time to release a new version of the software. Another complication that might occur when working on a project without a clear strategy is an opposite situation, when a team of developers frequently commits large chunks of new code to the main branch either by creating short-lived development/feature branches, or by committing directly to the main branch. When combined with being inconsistent about the use of branches and ambiguous commit messages, such a situation can make it very difficult to track and debug changes introduced to the main branch. Yet another scenario, that might be considered a risk to the project, is when there is no automated verification of the code merged into the main branch. This can lead to several problems like, for instance, adding too complex blocks of code, adding code with a lot of bugs or even adding a code that does not build or run project properly. Other risks regarding lack of strategy include issues such as: release unreliability, configuration drifts or low code review quality.

A vast majority of these problems can be mitigated with a surprisingly simple principle and it somewhat relates to the *divide-et-impera* approach. If we base our development strategy on the assumption that every developer integrates the code frequently by small increments, like fixing a bug or delivering a small feature, not only do our changes become easier to understand and debug, but also we reduce risks regarding unreliable code releases (famous “*works on my machine*”), hidden bugs, configuration drifts and many others. That is why Continuous Integration (CI) has become a standard practice for many IT developers worldwide. As defined in [9] “*CI ... is the practice of combining code changes frequently, where each change is verified on check-in*”. It “*ensures that coding changes are merged, compiled/linked, packaged and registered frequently, avoiding broken builds and ensuring a clean release candidate is available continuously*”. For example, let us say a developer A and a developer B work on separate features of the application and checkout their feature branches from the same commit from the main branch. Developer A follows CI best practices and has split his changes into two increments. Each increment has been verified by the team before being merged back into the main branch. In the meantime, a developer C has merged a large chunk

of code into the main branch, which has impact on the features developers A and B are developing. Developer B, who had been working on a long-lived branch with a larger set of changes, did not integrate the main branch updates as frequently. Developer A was able to easily resume working on a second increment of his feature, because they regularly updated their feature branch with the latest changes from the main branch. Meanwhile, developer B planned to merge their large feature update back into the main branch, but got stuck for several hours trying to fix merge conflicts and adapt their code to align with changes made by the developers A and C. There are several practical suggestions for developers aiming to improve their Continuous Integration. First suggestion would be to always try to split updates into batches small enough to integrate into the main branch at least once per day. While pushing changes frequently improves visibility, the key CI practice is frequent integration into the shared base branch, supported by automated verification. Another piece of advice would be to make absolutely sure the main development branch contains only healthy and reliable code. Several factors play crucial role in keeping the main branch without major problems. First of all, any change integrated into this branch should be reviewed according to an agreed policy (e.g., at least one or two peer approvals, or code-owner review for sensitive areas). Second of all, any change integrated into this branch should optimally be tested and validated holistically, for instance by code coverage, code clarity, unit tests and other programmatic testing solutions. This piece of advice is sometimes extended by more advanced security testing like penetration tests, secrets scanning, Dynamic Application Security Testing (DAST) or dependency vulnerability testing into what is called Continuous Testing (CT).

Going one step further, many IT projects require multiple complex steps to build and publish applications, and it soon became clear that even with a CI approach, performing these tasks manually after each integration can be tedious and time-consuming. Therefore, CI is often complemented by an automated approach such as Continuous Delivery (CD), so that developers do not need to spend significant time on repetitive build and release activities after each merge into the main branch. CD is a process of continuously (e.g. automatically) building artifacts from the main branch code, so that deploying application to staging or especially production environment becomes easy and repetitive step. A combination of Continuous Integration and Continuous Delivery creates an automated and repetitive process of creating application versions from merging updates into the main Git branch to successfully building production-ready application releases. Sometimes the CD acronym is confused with Continuous Deployment. As defined in [10] *“Continuous deployment is an approach where working software is released to users automatically on every commit. The process is repeatable and auditable”*. In other words, this is a process of deploying application to production in a continuous way. As this thesis focuses more on Continuous Delivery, Continuous Integration and Continuous Delivery (CI/CD) is the default expansion of this abbreviation. However it is very common in the industry to interchange Continuous Delivery with Continuous Deployment and doing so does not usually cause confusion in everyday communication, because the intended meaning is often clear from context. Apart from that, both approaches aim to keep the production-ready version of an

application buildable from integrated code. The key difference is whether the final release step is performed manually (Continuous Delivery) or automatically after successful verification (Continuous Deployment). Depending on the type of system, this “release” may mean deploying to a running environment (e.g., a web application) or publishing a packaged artifact for users to download or update (e.g., desktop or mobile applications).

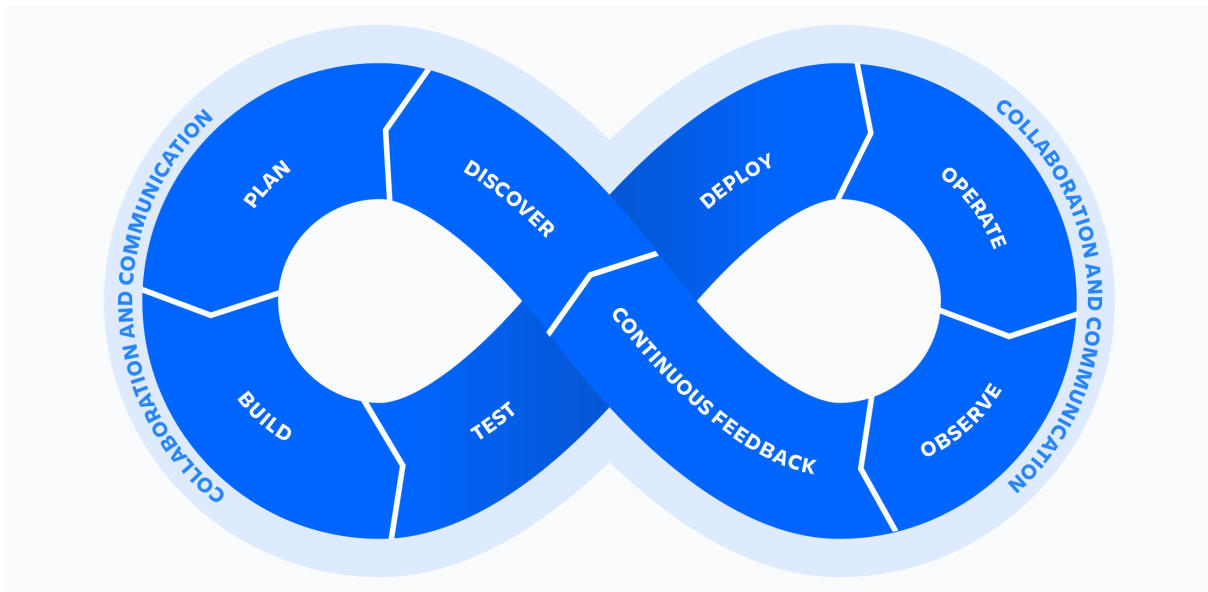


Figure 6. Graphical depiction of the perpetual DevOps software lifecycle (the infinity loop) [11]

CI/CD is a part of the software delivery lifecycle, which starts with an idea or plan and continues through development and release activities up to application operation, observability, and, eventually, version retirement (end-of-life). There is one particular IT projects methodology, that places software delivery lifecycle at its core and that methodology is called *DevOps*. The term DevOps originates from combining *development* with *operations* and this shows really well what DevOps is about. As defined in [12] “*DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams. It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.*”. Automation and integration of any processes in software delivery lifecycle are core principles of the DevOps methodology. This approach allows development teams to deliver new releases faster while assuring the operation teams that each new release remains reliable in the production environment. One very important theorem in the DevOps methodology is the DevOps infinity loop, presented in the Figure 6. It illustrates the software delivery lifecycle as a continuous chain of automated activities supported by feedback. Any application starts from some kind of a plan, which is later on built by a team of developers. The application code base is then automatically tested by special testing software and if it passes all tests it is automatically deployed to staging environment, which main purpose is additional testing and then on to the production environment. Once new release is deployed in a production environment, the production environment should be operated automatically (example of this could be

for instance configuring Disaster Recovery (DR) plans, secrets rotation or vulnerability management of the underlying infrastructure), for instance using Infrastructure as Code (IaC) approach. In parallel to automating operations for the application, the application itself should be automatically monitored using dedicated tools. Continuous feedback collected from the monitoring systems and also from end users plays a crucial role in discovering bugs in application or new possible areas of expansion. All collected responses should be evaluated and new versions of the application should be planned based on these responses.

The DevOps infinity loop is a theoretical representation of how a software delivery lifecycle should ideally look. It includes both technical steps and non-technical elements, such as feedback and human decision-making. Since not every part of this lifecycle can be automated, teams typically automate only the repeatable technical activities (e.g., build, test, packaging, and deployment) using what are commonly referred to as CI/CD pipelines. There are many different definitions of what pipelines in computer sciences are. However, for the sake of this thesis a DevOps pipelines definition is going to be the most useful. *“A DevOps pipeline is a combination of automation, tools, and practices across the SDLC² to facilitate the development and deployment of software into the hands of end users”* [13]. It is worth adding to this definition that steps in the DevOps pipeline are often defined by a chain of dependent stages in, where later stages do not start if any of the earlier stage fails – hence the term *pipeline*. In particular a *CI/CD pipeline* is a DevOps pipeline, that focuses primarily on the part of the DevOps infinity loop from the integration of the new software version into the main branch to building the production-ready version. Currently CI/CD pipelines are integrated with Git repositories so that when new version is published, code is being automatically linted, tested, then eventually deployed in staging environment for more in-depth testing (the optional CT phase) and if all previous steps succeed a production-ready application version is being built. CI/CD pipelines managed locally by popular systems like *Jenkins* [14] offer huge operational capabilities to configure automation that can even go beyond what is referred to as a CI/CD. However, they require large operational overhead and therefore, it is becoming more and more popular to make use of the CI/CD systems integrated within Git hosting providers, which offer CI/CD pipelines managed by the provider, thereby reducing the amount of operation needed to maintain underlying infrastructure, while still enabling teams to automate builds, tests, and releases. The next subchapter explains how these concepts have been incorporated into the most popular Git hosting platforms, namely GitHub, GitLab and Bitbucket as well as outline appropriate terminology each of these provider uses.

²SDLC: software development life cycle

2.3 Overview of major Git hosting platforms

The main purpose of this master's thesis is to perform a comparative analysis of CI/CD pipelines provided by GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines. The following subchapter presents all the terminology required to map general ideas to provider-specific nomenclature as well as brief introduction to each of these providers. Distributed Version Control Systems often make use of the remote server so that each local copy of the code repository can fully synchronise with the changes made to the remote branches – it works as a single source of truth. While some projects with restrictive security requirements or legacy projects may require Git server to be configured and maintained using self-hosted infrastructure, an overwhelming majority of modern IT projects use one of many available Git-hosting Software as a Service (SaaS) providers, which is very convenient as it eliminates the need to maintain the Git server.

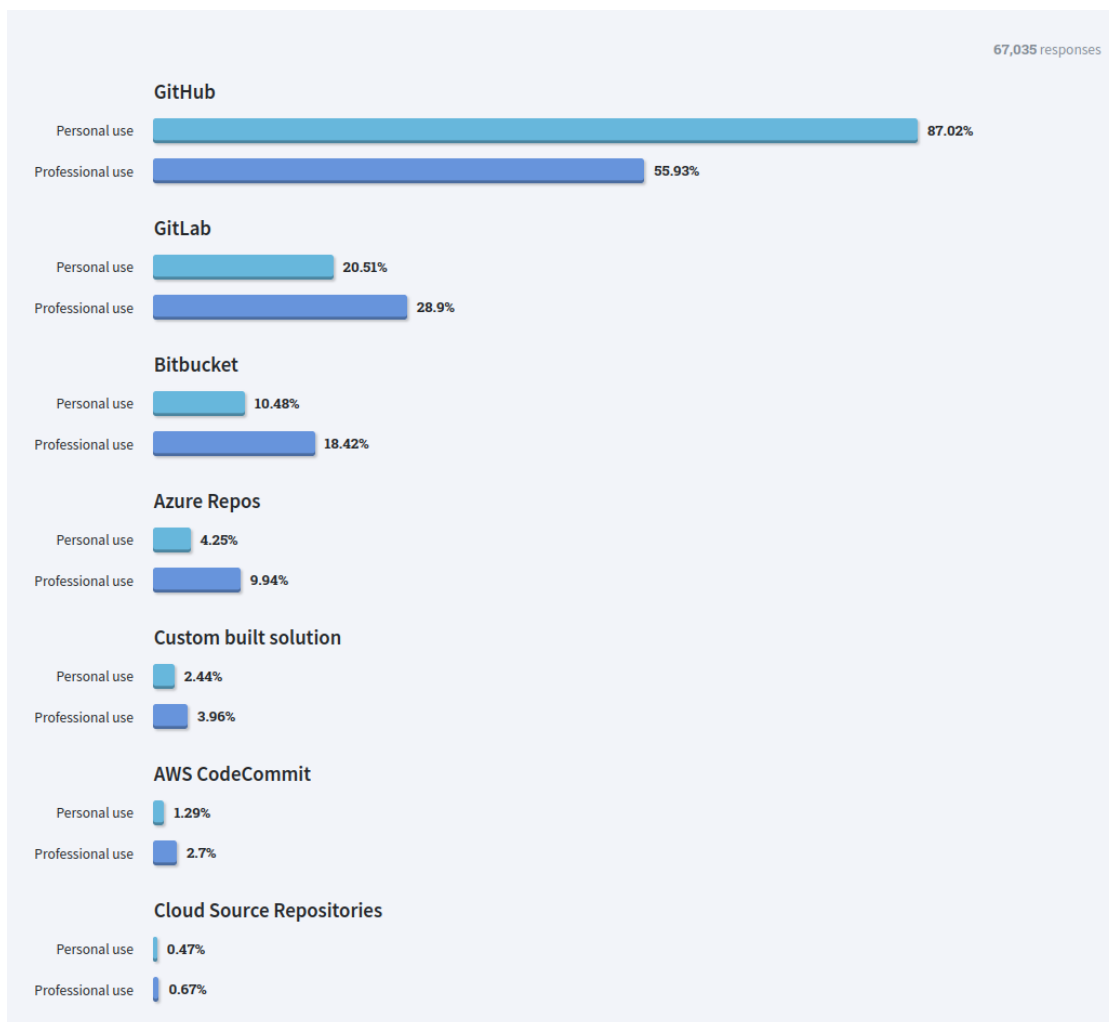


Figure 7. 2022 "Stack Overflow" Developer Survey – version control hosting service popularity [15]

According to the 2022 “*Stack Overflow*” Developer Survey [15], which results are presented in the Figure 7, respondents most frequently report using GitHub as their version control hosting service, both in personal and professional settings, while GitLab and Bitbucket are the next most prevalent alternatives, particularly in professional use. 67035 respondents have answered the question “*What version control hosting service are you using?*” in a following way – this was a multiple answers question. For personal projects developers most frequently use: GitHub (about 87%), GitLab (about 21%), and Bitbucket (about 10%), while for professional settings developers most frequently use: GitHub (about 56%), GitLab (about 29%) and Bitbucket (about 18%). It is worth noticing, that GitHub in this survey exhibits a marked dominance over the remaining platforms, with a large focus on personal projects. Many various reasons might have been the reason why GitHub is the most common VCS solution, however two options stand out. First of all, a plethora of online programming tutorials start by introducing GitHub as a place to version your code and as GitHub has been the starting point for many in the industry, it has also become the most widely used. Second of all, a lot of open-source projects are using GitHub as a version control platform and open-source solutions are used in almost any modern IT project, since they offer ready-to-go solutions free of charge. This survey clearly shows that GitHub, GitLab and Bitbucket dominate modern VCS hosting landscape.

According to the official documentation of the GitHub: “*GitHub is a cloud-based platform where you can store, share, and work together with others to write code*” [16]. Launched in April 2008 by Tom Preston-Werner, Chris Wanstrath, P. J. Hyett and Scott Chacon. Just within a year of it functioning, it has accumulated over 46000 public repositories with over 100000 users, which shows that the demand for such a product was very high at that time. Over the following years GitHub accumulated even more users and in 2018 it was acquired by Microsoft. Nowadays, GitHub has over 420 million repositories and over 150 million developers worldwide and is indisputable leader among the VCS hosting platforms. GitHub is available in five different versions. The first option that is most common for personal projects is the *GitHub Free* both for organisations and personal use. This version is free-of-charge and is enough to fully cooperate on IT projects using Git and has few standard features that allow teams to manage access, review and merge changes, track work items, and automate basic verification and delivery tasks. Another options are respectively *GitHub Pro* for personal use and *GitHub Team* for organisations. *GitHub Pro* costs 4\$/month [17] – as of 2025 – and it offers additional features like exempli gratia GitHub Support via email, 3000 minutes³ per month for GitHub Actions (which is further described in this subchapter), advanced tools in private repositories. *GitHub Team* costs 4\$/month [17] – as of 2025 – and it offers similar features like *GitHub Pro* but scaled for the needs of an organisation and additionally *GitHub Advanced Security* products may be additionally purchased with this plan, which includes GitHub Code Security and GitHub Secret Protection. The last two versions of GitHub available regard only organisations. The first of them is *GitHub Enterprise Cloud*, which offers the most options, including GitHub Enterprise Support, GitHub Connect, Authentication with SAML single sign-on, Access provisioning with SAML

³GitHub Actions minute limit applies only to private repositories

or SCIM, 50000 minutes per month for GitHub Actions and other features highly valued by large organisations and enterprises seeking to host their IT projects in GitHub. The last of the GitHub pricing options is *GitHub Enterprise Server* which offers similar benefits like *GitHub Enterprise Cloud* with this exception, that *GitHub Enterprise Server* is a GitHub instance that is hosted and managed locally in an enterprise environment – particularly in an on-premises environment. This option is designed for organisations developing code, that has to reside within company's infrastructure, which comes with an additional benefit that this option provides unlimited amount of minutes for GitHub Actions, since now GitHub Actions runners are hosted on an organisation's environment. *GitHub Enterprise Cloud* costs 21\$/month [17] with a 30-day free trial option – as of 2025 and *GitHub Enterprise Server* is charged individually depending on the deal a company makes with the GitHub sales department. This thesis tests capabilities of *GitHub Actions*, which is a service that manages CI/CD pipelines in GitHub and this option is available in all plans, therefore the default version of GitHub analysed in this thesis is the *GitHub Free* version and even if the limit of 2000 minutes per month of GitHub Actions with GitHub-hosted runners is exceeded and higher plans must be considered to generate more test results, this version update does not impact the actual test results in any way. For that reason this subchapter explains few selected features available in *GitHub Free* plan and therefore available in any higher plan.

GitHub repository is a place where the project's Git repository – that is code base – resides. Each repository can be either private or public. Public repositories are visible to anyone on the Internet, whether logged in to GitHub or not. This is the best option for individuals seeking to for instance showcase their personal projects or for organisations to share their open-source solutions. GitHub private repositories on the other hand are visible only to the repository owner or team members in higher plans. It offers the best solution for copyrighted or protected projects. A GitHub repository can either belong to a user or to a GitHub organisation, which is a way to group repositories together under *GitHub Free* for organisations. Any GitHub user can create GitHub organisations and in *GitHub Enterprise* plans GitHub organisation can belong to a GitHub enterprise. GitHub enables following options for its repositories. First of all, GitHub has created a way for proposing and merging changes between branches on the hosting platform, known as a “pull request”. This feature supports collaboration by enabling peer review and discussion through inline comments, and it also allows automated CI checks (e.g., linting and testing) to be executed before changes are merged into the target branch. Repositories in GitHub contain features, such as “issues”, which is available only for public repositories and allows any other GitHub user to post potential issues with the code, like bugs noticed or possible room for improvement. Other features in GitHub include the *Releases* and *Packages* sections, which provide a central place to browse published application versions and release notes, as well as to access artifacts produced during the CD phase.

The most important option from the perspective of this thesis is GitHub Actions, which is a place to view, debug or monitor CI/CD pipelines run in the repository. GitHub Actions dashboard shows

a list of historic CI/CD pipelines runs, which are called “*workflow runs*”. Each workflow in GitHub Actions contains jobs, which are collections of steps executed in a defined order on the same runner to perform a specific task, such as building, testing, or deploying the application – a job can have multiple steps. A GitHub Action workflow is defined in a YAML manifest file, which must be present in the `.github/workflows` directory in the repository. As defined in [18], “*YAML is a human-friendly data serialisation language for all programming languages*”. This language nests elements by the indentation level in a Python-style and is very common for configuring automation and infrastructure tools, including CI/CD workflow definitions and pipeline configuration files.

```
1  ---
2  name: Simple CI (matrix + needs)
3  on:
4    push:
5      branches: [ "main" ]
6    pull_request:
7      branches: [ "main" ]
8
9  jobs:
10   test:
11     name: Test (matrix)
12     runs-on: ubuntu-latest
13
14     strategy:
15       fail-fast: false
16       matrix:
17         node: [ "20", "22" ]
18
19     steps:
20       - name: Checkout code
21         uses: actions/checkout@v4
22
23       - name: Setup Node.js ${ matrix.node }
24         uses: actions/setup-node@v4
25         with:
26           node-version: ${ matrix.node }
27
28       - name: Install + test
29         run: |
30           npm ci
31           npm test
32
33   build:
34     name: Build (after tests)
35     runs-on: ubuntu-latest
36     needs: test
37
38     steps:
39       - name: Checkout code
```

```
40     uses: actions/checkout@v4
41
42   - name: Build
43     run: |
44       npm ci
45       npm run build
```

Listing 1. Example GitHub Actions workflow in YAML

Listing 1 presents a simple example how GitHub Actions are defined using a *YAML* manifest. A GitHub Actions workflow requires two elements, defined at the top level of this file and at least one job defined. The `on:` object defines the trigger on which the workflow must run – in this particular example the workflow runs if someone pushes changes to the branch “*main*” or creates, or updates, a pull request to the branch “*main*”. Next, the `jobs:` object is a place where jobs for the workflow are configured. The child element of the `jobs:` object is always a job id, which can later on be used as a reference in the workflow. This element can contain a lot of options, fully described in the documentation [19], however for the sake of simplifying explanation following standard elements are almost always necessary to define a job in GitHub Actions. First is the `name:` variable, which is a human readable name of the job, that can contain whitespace characters and is visible later in GitHub Actions dashboard. Second variable is `runs-on:` and it tells GitHub Actions whether to run this jobs using GitHub-hosted runners, or self-hosted runners. In this example `ubuntu-latest` tells GitHub Actions to use GitHub-hosted Ubuntu-based⁴ runner. Next, an optional object `strategy:` is an instruction for GitHub Actions on how exactly to run this job – in this case a child object `matrix:` defines a list of values under the `node:` variable and GitHub Actions run this job in parallel for each element in this list. The last, but not least is the `steps:`, which is an object under `jobs:<name>:` and defines a list of tasks this job has to perform. There are a plethora of options on how to define tasks in a workflow job, but the workflow in the Listing 1 contains only the most common ones. Each step has to define a human-friendly name under `name:` variable. Other options include `uses:`, which is a GitHub Actions option to use community actions or reusable workflows – in particular other *YAML* manifests – and `run:` variable, which is a way in GitHub Actions to run shell commands on the runner machine.

GitLab was created in 2011, as a side project to make collaboration on code easy and efficient, by Dmitriy Zaporozhets and Valeriy Sizov. GitLab was originally an open-source software forge which meant that its code base was open-source and anyone could contribute to its development, which was a crucial factor for the platform to gain popularity in its early years. GitLab is available in three different modes of operation [20]. The first is the *GitLab.com* version which is fully managed by GitLab cloud hosted instance and is the most popular choice for many developers, including some enterprise teams. This version is enough to be able to fully cooperate on the IT project using Git VCS.

⁴Ubuntu is one of the most popular Linux operating system distributions

The second option is the *Self-Managed* version, which is a forge instance of the GitLab, which can be run locally and is a great option for full control over project's data, but requires managing underlying infrastructure. The last mode is the *Dedicated* mode, which is a single-tenant SaaS fully managed by GitLab and offers the most when it comes to providing security and compliance with the most demanding rules or regulations regarding the data stored in such repositories. Both *GitLab.com* and *Self-Managed* versions come in two pricing plans. The first is the *Free* plan, which for cloud version offers features like 5 licensed users, 400 compute minutes per month – for CI/CD pipelines and 10 GiB of storage along with some paid add-ons. The second pricing plan is the *Premium* plan, which comes with additional features like release controls, Service Level Agreement (SLA) management or 10000 compute minutes per month and costs 29\$/month [20] – as of 2025. Pricing for dedicated GitLab instances is negotiated individually with the GitLab sales department. As this thesis focuses on comparing capabilities of CI/CD pipelines across the three vendors, further GitLab description is based around the *Free* version and in this case generating more test results may only require buying additional compute for CI/CD pipelines without ever upgrading the pricing plan.

In GitLab, Git repositories are organised as *projects*, whereas in GitHub they are referred to as repositories. In practice, both concepts represent the same core unit: a remotely hosted Git codebase with a web interface for collaboration. A GitLab project can be public or private, similarly to GitHub. Projects may belong to an individual user (personal projects) or to a GitLab *group*. A GitLab group serves a role comparable to a GitHub organization by providing a shared namespace for multiple projects and enabling centralized user management and access control. The main differences start to appear when we consider details of how Git repositories are organised within GitLab project and GitHub repository respectively. The first major difference is how GitLab designed its UI navigation panel. It organises project settings and features into clear DevOps phases that are a part of the DevOps infinity loop, which shows that GitLab implicitly encourages users to follow this popular methodology. The navigation panel begins with the *Pinned* section, which is placed there just for convenience of the developers, followed by the *Manage* section, which focuses on user and access management and is crucial for granting, restricting, or revoking permissions for specific users. From this point onward (except for *Settings*), the navigation follows the DevOps/SDLC phases. The *Plan* options contain features like *Issues*, which works like Issues in GitHub, or *Milestones* – both great for project management and planning phase. The *Code* option contains all the Git-related elements like branches, commits, and tags, and *Merge Requests*, which correspond with GitHub's Pull Requests. The following three options, namely *Build*, *Secure* and *Deploy* all focus on the CI/CT/CD part of the DevOps/SDLC, with *Build* containing CI/CD pipelines configuration options, *Secure* containing security and auditing features and *Deploy* includes releases and registries. Options *Operate*, *Monitor* and *Analyze* correspond with the operate, monitor and analyse parts of the lifecycle.

As far as the CI/CD pipelines in GitLab are concerned, they are called *Pipelines* and can be viewed in *Build/Pipelines* dashboard. Each pipeline in GitLab contains stages, which as the name suggests,

allows to group pipeline jobs in stages. The stages are linked together in a chain of dependencies based on the order in which they are defined in the file. All GitLab pipelines are defined in a single YAML manifest file named `.gitlab-ci.yml`, which must be located in the project root.

```
1  ---
2  stages:
3    - testing
4    - build
5  ##### Stage: test #####
6  # ---- Job: unit_tests ----
7  unit_tests:
8    stage: testing
9    image: node:${NODE_VERSION}
10   parallel:
11     matrix:
12       - NODE_VERSION: ["20", "22"]
13   script:
14     - npm ci
15     - npm test
16  # ---- Job: lint_check ----
17  lint_check:
18    stage: testing
19    image: node:22
20    script:
21      - npm ci
22      - npm run lint --if-present
23  ##### Stage: build #####
24  # ---- Job: build_app ----
25  build_app:
26    stage: build
27    image: node:22
28    script:
29      - npm ci
30      - npm run build
```

Listing 2. Example GitLab CI/CD pipeline in YAML

Listing 2 presents a way pipelines are defined in GitLab CI/CD using a YAML file, that has to be named exactly `.gitlab-ci.yml`. Comments, denoted by `#` characters, have been added to clearly divide file into stages and jobs. At the top of this file, a list of stages can be added – in this particular example two stages: `testing` and `build`. The rest of the file contains job definitions or objects on the top level. A `.gitlab-ci.yml` file has to have at least one such object defined. In the `testing` stage there are two job objects: `unit_tests` and `lint_check` and each of them contains `stage:` variable, which links jobs into stages. If `stage` is not defined in the job, then it belongs to the default stage. In each job `image:` variable must be defined – with the exception of running jobs on self-hosted environments – so that GitLab runner could pull the right Docker image and execute

job using specific container [21]. Other job variables are non-mandatory, however two most common variables have been added to the example in the listing 2, namely a `script:` variable, which is a list of terminal commands to be run inside the job container and a `parallel` with its child `matrix` object, which in this example contains a list of variables named `NODE_VERSION:` and is used for job parallelisation. For instance, job `unit_tests` is run twice for each value in `NODE_VERSION`.

Bitbucket.org was an independent code-hosting service focused on the Mercurial DVCS, led by CEO and co-founder Jesper Nøhr. Atlassian company acquired Bitbucket.org on 29 September 2010 [22]. Later it expanded to Git hosting, becoming a major competitor among source code hosting platforms. One of the most important reasons of the popularity of Bitbucket is the fact that it is Atlassian solution fully integrated with other Atlassian tools – in particular with the Jira software, which is currently one of the most popular choices for IT project management. As a result, Bitbucket is often adopted by organisations that already rely on the Atlassian suite of tools and seek a tightly integrated workflow for planning, issue tracking, and source code management. Bitbucket comes with three pricing plans: a Free plan, a Standard plan and a Premium plan. The Free plan is available only for teams up to 5 Atlassian users, but it offers some key features, such as unlimited private repositories and pull request reviewers, merge checks, Jira integration and more [23]. Apart from that, the Free version has a total workspace storage limit of 1 GB and a 50 minutes per month limit on CI/CD builds, which can be increased the same way as in GitLab – that is 10\$ per 1000 additional build minutes. The Standard plan is designed for teams that require higher limits than the Free tier and is billed in user tiers. The price for this plan is billed depending on the number of the users involved. Starting at 18.25\$ per user per month for 1 user and going down to 3.10\$ per user per month for teams of 35000 users [23] – as of 2025. For both Standard and Premium plans the price for teams exceeding 35000 users is negotiated with the Bitbucket sales department. It retains the core collaboration features that exist in the Free plan while increasing included resources, in particular by providing 2500 CI/CD build minutes per month, no workspace storage limit, AI-enhanced comments and summaries and standard Bitbucket support. Atlassian recommends the Standard plan for teams of more than 5 users. Above the teams of 100 users Atlassian recommends the Premium plan. It is the highest available Bitbucket plan and it features the same options as the Standard plan with some additional options, such as 3500 CI/CD build minutes per month, 2 build slots for self-hosted runners, dynamic CI/CD pipelines, 6 additional security features and premium Bitbucket support. The pricing for this plan works in the same way it does for the Standard plan but with higher prices starting at 36.25\$ per user per month for 1 user and going down to 6.06\$ per user per month for teams of 35000 users [23] – as of 2025. The most popular option for the Bitbucket is the cloud-hosted `Cloud` version, however Bitbucket offers also the `Data Center` version, which is a self-hosted version of the Bitbucket that can be deployed on-premises. As this thesis focuses on comparison of capabilities of the CI/CD pipelines, technical features present in the `Cloud Free` version are enough for the CI/CD pipelines to be objectively compared with GitHub's and GitLab's solutions.

Git repositories are stored in Bitbucket as repositories unlike in GitLab. Bitbucket repositories can be either private or public, just like in GitHub and GitLab. However, unlike in GitHub, Bitbucket can group repositories in projects, which can be useful for managing multiple repositories in an workspace. A Bitbucket workspace can be compared to GitHub's organization and GitLab's group, and can be created by an Atlassian account. Repository navigation in Bitbucket is done in a similar manner as in GitLab – that is through a left side bar with options. However, this side bar does not follow the DevOps/SDLC phases as visibly as in GitLab, yet some similarities remained. The first three options regard only Git-specific code base management features. The Source option is a starting view of the Bitbucket repository and it allows browsing the repository contents, including files and directories on any Git branch. The next view is named Commits and it makes it very intuitive to analyse commit history of each branch in the repository by viewing Git commit history graph along a commit data table. The third Git-specific option is the Branches view, which shows full history of branches in the repository including Git tag placement and other useful insights. Next three options in the navigation bar are related to running CI/CD pipelines. The first option in this “section” is the Packages option, which provides access to package artifacts associated with the repository (e.g., container images or language-specific packages), allowing teams to publish and consume build outputs in a central registry. The second option is the Pipelines view, which shows a table containing full history of pipeline runs in this repository with detailed information on the exact commit that triggered the run, exact branch, pipeline, run status and many more. This view works similar to how it is designed in GitHub and GitLab, allowing users to expand each pipeline run to view detailed logs. The last option regarding running pipelines in Bitbucket is the Deployments option, which shows deployment-specific information from running deployments in CI/CD pipelines – Bitbucket selects only crucial information from the pipelines file based on the deployment keyword. Next option really shows why Bitbucket is often chosen by teams who manage their projects with the Jira software. The Jira work items bar integrates issue tracking directly into the repository view by linking commits, branches, and pull requests to the corresponding Jira issues, which improves traceability between project planning and code changes. Next, Bitbucket repository view is the Security view, which allows team to leverage capabilities of the Snyk tool which, as defined in the official documentation, is “*a platform that allows you to scan, prioritize, and fix security vulnerabilities in your code, open-source dependencies, container images, and infrastructure as code configurations*” [24]. Next option is an additional bar present only paid Bitbucket plans, namely the Downloads bar which allows to place there files that can be downloaded, such as the compressed archive of the code repository. However, this option is irrelevant from the perspective of performance analysis of CI/CD pipelines and its non-existence in the Free plan does not impact any further test results. The last element in the navigation bar is the Repository settings option, which just groups all available settings of the Bitbucket repository, such as access tokens, branch restrictions, SSH keys and other options. As far as the CI/CD pipelines are concerned, in Bitbucket they are called pipelines and are managed from the Pipelines option. Each pipeline run information is stored there and this information shows pipeline status and logs for each step defined in the definition file. In Bitbucket all

pipelines for the repository must be defined in a single YAML file named `bitbucket-pipelines.yml` which has to be placed in the repository root.

```
1  ---
2  image: node:22
3  definitions:
4    steps:
5      - step: &unit_tests
6        name: Unit tests
7        script:
8          - npm ci
9          - npm test
10     - step: &build_app
11       name: Build application
12       script:
13         - npm ci
14         - npm run build
15 pipelines:
16   branches:
17     main:
18       - parallel:
19         steps:
20           - step:
21             <<: *unit_tests
22             name: Unit tests (Node 20)
23             image: node:20
24           - step:
25             <<: *unit_tests
26             name: Unit tests (Node 22)
27             image: node:22
28
29       - step: *build_app
30 custom:
31   simple-ci:
32     - parallel:
33       steps:
34         - step:
35           <<: *unit_tests
36           name: Unit tests (Node 20)
37           image: node:20
38         - step:
39           <<: *unit_tests
40           name: Unit tests (Node 22)
41           image: node:22
42     - step: *build_app
```

Listing 3. Example Bitbucket pipelines YAML definition

Listing 3 presents an example Bitbucket pipeline definition in `bitbucket-pipelines.yml` file. All pipelines regarding the repository are defined in this single file under the `pipelines:` object. Bitbucket reads all children objects of the `pipelines:` object and determines how to trigger each pipeline. In this example the first pipeline is triggered on push to the main branch – this is defined by the `branches:main:` object. The second pipeline is triggered manually – or programmatically via Bitbucket’s Application Programming Interface (API) – and all manual pipelines are defined by the `custom:` object. This pipeline holds the name defined by the most direct `custom:` descendant, the `simple-ci:` object. Bitbucket divides pipeline primarily into `steps`, which can be compared to GitHub’s and GitLab’s `jobs`, with the exception of the parallel blocks, which are discussed further in this paragraph. In both example pipelines, either `main:` object or `simple-ci:` objects are defined as a list of such steps. Some steps can be however defined in many versions and each run in parallel. For this specific purpose, list element named `parallel:` can be used, where steps defined in an `steps:` object are run in parallel. This Bitbucket pipeline definition file makes use of the YAML anchors, which are a standard feature of the YAML language and allow for reusing the same objects within the same file. This feature is especially useful with Bitbucket pipelines, as very often teams require large number of different pipelines run in the same repository and creating templates makes this single file easier to read and debug. Those template steps are defined in the list stored in the `definitions:steps:` object. Each step contains `name:` and `script:` variables, which define respectively a step name and a list of shell commands to be run and each reusable step is marked by the YAML anchor, which is denoted by the `&` character. Those `&` anchors can be later referenced by steps in the `pipelines` section using the `*` character. Reusable steps can be customized when they are invoked. In the parallel block, the YAML merge key `<<:` imports the anchored step definition, and any keys defined alongside it (e.g., `name:` or `image:`) override the imported values. As far as the `image:` variable is concerned, it is a variable that is required to be defined for each step in Bitbucket pipelines. However, this and few other variables can be defined at the top level of the YAML file, so that they serve the purpose of the default variables. To summarise this subchapter, the table below is created to consolidate the key terminology and map the most important concepts to the provider-specific nomenclature used by GitHub, GitLab, and Bitbucket.

Table 1. Key nomenclature differences between GitHub, GitLab, and Bitbucket

Concept	GitHub	GitLab	Bitbucket
Edition / mode names	GitHub Cloud, GitHub Enterprise Cloud, GitHub Enterprise Server	GitLab.com, Self-Managed, Dedicated	Cloud, Data Center
Plan names (within the above)	GitHub Free / Pro / Team / Enterprise (as above)	Free, Premium (for GitLab.com and Self-Managed)	Free, Standard, Premium
Primary code container term	Repository	Project	Repository
Main grouping term above code container	Organization (optionally within an Enterprise)	Group	Workspace (can group repositories into Projects)
Extra grouping layer mentioned	Enterprise (above organisations, in Enterprise plans)	None	Project (groups repositories within a Workspace)
Change proposal / merge term	Pull request	Merge request	Pull request
Issue/work tracking term mentioned	Issues	Issues, Milestones	Jira work items (integration)
CI/CD product name	GitHub Actions	GitLab CI/CD	Bitbucket Pipelines
Pipeline definition unit (top-level)	Workflow	Pipeline	Pipeline
Execution unit inside pipeline	Job	Job (grouped into Stages)	Step (parallel blocks contain multiple steps)
Execution agent term	Runner	Runner	Runner (self-hosted runners use build slots)
Pipeline definition file name	any	.gitlab-ci.yml	bitbucket-pipelines.yml
Pipeline definition file location	.github/workflows/ directory	Project root	Repository root

2.4 Introduction to the queueing theory

The purpose of this master's thesis is to conduct a comparative analysis of CI/CD pipelines. In particular the research conducted in this thesis utilises definitions from the field of mathematics, known as the queueing theory. As this is a broad field of study, this subchapter introduces only the most important definitions needed to understand calculations performed on the collected dataset and understand, how these results are interpreted. Queueing theory is a field of mathematics that analyses and describes different parameters of queues – or waiting lines – for one of the many queue models. Queueing theory has a plethora of applications in all various fields of study, however in computer sciences it is used mostly to analyse message queueing systems, which at some extent could be applied to running CI/CD pipelines in GitHub, GitLab and Bitbucket. This thesis explores that aspect in more detail.

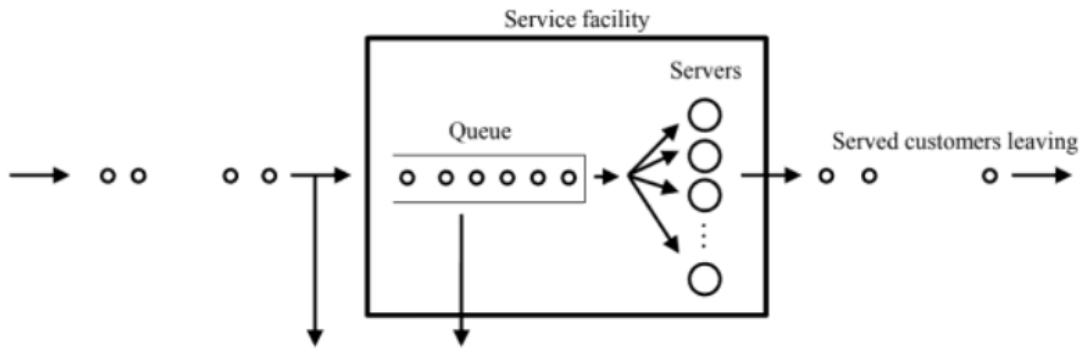


Figure 8. General queue model [25]

A general model of any queue is depicted in the Figure 8. The first element is a customer, which is an entity arriving to the system that may need to wait in a buffer before being served by the available server(s). In this model server is performing some job for each customer it serves. As far as the general queueing theory nomenclature is concerned, following measurements are particularly useful from the perspective of this thesis. Arrival time t_i for i -th customer, respective inter-arrival time $A_i = t_i - t_{i-1}$ and expected (mean) arrival rate $\lambda = 1/\mathbb{E}[A]$ calculated from expected inter-arrival time $\mathbb{E}[A]$ and service start time s_i , and service completion time d_i . This allows to define formulas for waiting time in queue and service time respectively: $W_{q,i} = s_i - a_i$ and $S_i = d_i - s_i$. A notation was developed in 1953 by David George Kendall to better describe any queueing system based on its mathematical characteristics. A short Kendall's notation uses three characteristics to deal with this task, namely $A/S/c$. The first letter refers to the inter-arrival time characteristics, the second letter refers to the service time distribution and the last letter denotes the number of parallel service channels – or the number of servers picking customers from the queue. Short Kendall's notation assumes the remaining three parameters are: $K \rightarrow \infty$ - queue capacity tends to an infinity, $N \rightarrow \infty$

- size of population of customers tends to an infinity and $D = \text{FIFO}$ - queue discipline is First In First Out (FIFO). The FIFO queue discipline means that customers are "ordered" in a queue based on the chronology of their arrival - the first customer is going to be served first. Both arrival time process and service time distributions are most commonly either M or G. The symbol M stands for *Markovian*. This means the parameters have *Markov property*, where process has no memory beyond the current state – is *memoryless*. In practice this means that arrivals follow a Poisson process. This is a stochastic process $N(t)$, for time $t \geq 0$ with rate of $\lambda > 0$, that has following probabilistic properties [25]. N denotes the number of arrivals that occurred in the time interval $[0, t]$.

1. $N(0) = 0$, meaning for time $t = 0$ number of customers arrived is 0
2. $\Pr\{N(t + \Delta t) - N(t) \geq 2\} = o(\Delta t)$, meaning that in a very short time interval $(t, t + \Delta t]$ the process can produce 2 or more arrivals with probability o dependent on the time interval duration
3. $\Pr\{N(t + \Delta t) - N(t) = 1\} = \lambda \Delta t + o(\Delta t)$, meaning that in the same time interval the process produces one arrival with probability proportional to the interval length
4. for disjoint intervals, the increments are independent.

In this definition, $o(\Delta t)$ denotes a quantity that becomes negligible when compared to Δt as $\Delta t \rightarrow 0$; that is [25],

$$\lim_{\Delta t \rightarrow 0} \frac{o(\Delta t)}{\Delta t} = 0. \quad (2)$$

In other words, events in this model occur randomly over continuous time at an average rate of λ and is extremely useful to describe independent, unsynchronised triggers generated by many sources, e.g., CI/CD pipeline runs initiated by multiple developers, repositories, or automated tools, where the aggregate arrival stream can be treated as approximately Poisson. The symbol G stands for a *general* (arbitrary) distribution and is extremely useful for describing non-Poisson system characteristics (e.g., bursty traffic, periodic triggers with jitter, or workload correlations). In the context of this thesis, three various queueing models are analysed for CI/CD pipelines. The first model is the single-server queueing model with Markovian (Poisson) arrivals, and general service-time distribution (M/G/1) model, which is useful for analysis of pipeline runs, that occur at a random rate, but in a continuous manner. The second model is the single-server queueing model with a general (arbitrary) arrival process, and general service-time distribution (G/G/1) model, which is useful for analysis of pipeline runs, that occur in an undefined pattern over a longer time window with occasional bursts and periods without pipeline triggers. The last model analysed is the multiple-server queueing model with a general (arbitrary) arrival process, and general service-time distribution (G/G/c) model, which has

the same characteristics of the G/G/1 model, with the only difference of assuming multiple servers serving the queue.

From the perspective of this thesis, the most important is the formula for expected (mean) wait time in queue $\mathbb{E}[W_q]$. Under the assumptions of M/G/1, derivations from the [25] lead to the final formula, known as the *Pollaczek-Khintchine* (PK) formula:

$$\mathbb{E}[W_q]_{M/G/1} = \frac{1 + C_S^2}{2} \cdot \frac{\rho}{1 - \rho} \cdot \mathbb{E}[S] \quad (3)$$

Where:

1. $\rho = \lambda \mathbb{E}[S]$ - is a single server utilisation for arrival rate λ and expected service time $\mathbb{E}[S]$
2. $\mathbb{E}[S]$ - is expected (mean) service time
3. $C_S^2 = \frac{\sigma_S^2}{\mathbb{E}[S]^2}$ - is squared coefficient of variation (SCV) of a service time S where σ_S^2 is a variance of the service time S

For G/G/1 and G/G/c models, as we have general – or unknown – characteristics of a queue, we can only approximate the expected wait time in queue $\mathbb{E}[W_q]$. The approximation for the G/G/1 model was presented by Sir John Frank Charles Kingman in 1960 [26]. The practical considerations from the *Kingman's* formula regard approximating the G/G/1 model with the expected wait time formula from the M/G/1 model.

$$\mathbb{E}[W_q]_{G/G/1} \approx \mathbb{E}[W_q]_{M/G/1} \cdot \frac{C_A^2 + C_S^2}{1 + C_S^2} \quad (4)$$

Where:

1. $\mathbb{E}[W_q]_{M/G/1}$ - is expected wait time in queue of the M/G/1 queue model
2. C_A^2 - is squared coefficient of variation (SCV) of inter-arrival times A
3. C_S^2 - is squared coefficient of variation (SCV) of service times S

For G/G/c models, a commonly used practical approximation is the *Allen–Cunneen* formula, which generalises the *Erlang-C* (M/M/c) baseline model by scaling the expected queue length by the average variability of inter-arrival and service times. This model assumes that service times are exponentially distributed, meaning the duration of serving a customer is random and memoryless - the chance of finishing soon does not depend on how long the service has already lasted.

The probability that an arrival has to wait in the $M/M/c$ queue is:

$$P_1 = \frac{\frac{a^c}{c!} \cdot \frac{c}{c-a}}{\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c!} \cdot \frac{c}{c-a}}. \quad (5)$$

Where:

1. $a = \frac{\mathbb{E}[S]}{\mathbb{E}[A]}$ - is a single server workload where $\mathbb{E}[S]$ is expected service time and $\mathbb{E}[A]$ is expected inter-arrival time
2. c - is the number of servers processing customers in the queue

The expected wait time in $G/G/c$ queue is therefore approximated using following formula.

$$\mathbb{E}[W_q]_{G/G/c} \approx \frac{P_1 \rho}{\lambda(1-\rho)} \cdot \frac{C_A^2 + C_S^2}{2} \quad (6)$$

Where:

1. P_1 - is the probability that an arrival has to wait in the $M/M/c$ queue
2. $\rho = \lambda \mathbb{E}[S]$ - is a single server utilisation for arrival rate λ and expected service time $\mathbb{E}[S]$
3. $\lambda = \frac{1}{\mathbb{E}[A]}$ - is customer arrival rate, where $\mathbb{E}[A]$ is expected inter-arrival time
4. C_A^2 - squared coefficient of variation (SCV) of inter-arrival times A
5. C_S^2 - squared coefficient of variation (SCV) of service times S

In particular formulas for $\mathbb{E}[W_q]_{M/G/1}$, $\mathbb{E}[W_q]_{G/G/1}$ and $\mathbb{E}[W_q]_{G/G/c}$ are used to analyse pipeline call latency and to answer a question if queueing theory can be applied for CI/CD pipeline description purposes.

Chapter 3

Solution implementation overview

This chapter describes the design and implementation of the experimental environment used in this thesis. Its role is to translate the theoretical concepts and models defined in the previous chapter into a reproducible setup that allows systematic measurement and comparison of CI/CD pipelines on the Git hosting platforms.

In the beginning, the chapter defines the planned architecture for the experiments and the criteria for making the environments on GitHub, GitLab, and Bitbucket comparable. Then, it presents the configuration of organisations or groups on each platform and describes the creation of two source code repositories together with an additional repository that stores CI/CD pipeline definitions. The chapter explains how the pipelines are constructed to resemble patterns from commercial projects, such as compilation, automated tests, and simple analysis tasks, and how additional steps are added to record metrics required for later evaluation. Next, it describes the configuration of managed runners, where applicable, and focuses on the deployment of self-hosted runners in a Kubernetes cluster running in the AWS cloud. Finally, it presents all the tools used for data collection and processing, with emphasis on the data pipeline implemented in the AWS cloud.

The material in this chapter ensures that the experimental procedure is transparent and can be reproduced or extended. It provides the basis for the results presented in the next chapter, where the collected data are analysed in terms of performance, queueing behaviour, scalability, and the differences between managed and self-hosted CI/CD runners.

3.1 Management repository and provider organisation configurations

Following subsection presents a way repository environment has been configured in a repeatable and provider-agnostic manner using Infrastructure as Code. The core idea is to define all organisations, projects, and source code repositories declaratively, so that an identical structure for GitHub, GitLab, and Bitbucket can be recreated at any point in time without manual intervention. This approach ensures that any future changes to the testing setup, such as adding a new repository or adjusting permissions, are applied consistently across all providers by updating a single set of IaC definitions and reapplying them. The general architecture diagram is placed in the Figure 9.

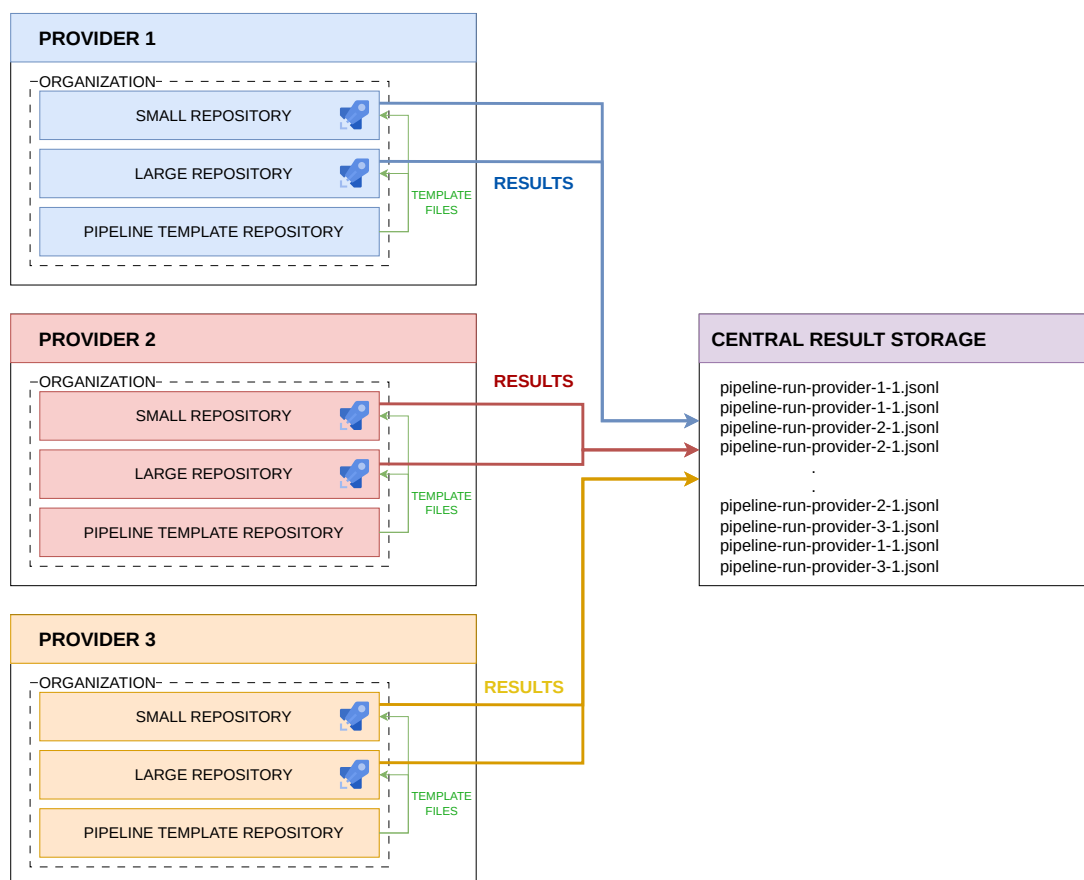


Figure 9. General testing environment architecture

The general architecture of the testing environment assumes that each provider hosts an identical organisation or comparable container, that has two source code repositories and one repository with shared pipeline templates. Each provider organisation must have the same small and large code base repositories, where main pipelines are executed. All metrics produced by the pipelines are exported to an external storage location in a unified JSONL format for further analysis. JSONL stands for *JavaScript Object Notation Lines* and it is a data storage format that consists of JSON objects in each line of the file. For every provider, both the small and large repositories reference

the same logical set of template definitions, so that pipeline definitions are easier to maintain and that any change to the pipeline structure is propagated consistently across the whole environment, regardless of where the pipeline is executed. At the end of each pipeline run, a separate result file is generated and pushed to a central result storage, which enables cross-provider analysis of performance characteristics. All the resources needed to implement this idea have been defined in a central (management) repository using IaC approach, which means that the infrastructure is provisioned as much as possible using code. This management repository is present in GitHub under the link: <https://github.com/Krzychu-Z/ci-cd-comparison> and all other instructions related with this subchapter can be found there.

```
.
├── environment
│   ├── bitbucket-arc.sh
│   ├── eks.tf
│   ├── github-arc.sh
│   ├── gitlab-arc.sh
│   ├── outputs.tf
│   ├── s3.tf
│   ├── terraform.tf
│   └── variables.tf
├── LICENSE
├── organizations
│   ├── bitbucket-org
│   │   ├── bitbucket_config.py
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── repo_functions.py
│   │   ├── requirements.txt
│   │   └── venv
│   ├── github-org
│   │   ├── organization.tf
│   │   ├── repos.tf
│   │   ├── terraform.tf
│   │   └── variables.tf
│   └── gitlab-org
│       ├── group.tf
│       ├── repos.tf
│       ├── terraform.tf
│       └── variables.tf
├── README.md
├── requirements.txt
└── run.py
```

Figure 10. Management repository file tree

The Figure 10 shows full repository structure of the management repository. It contains environment/ and organizations/ directories, a Python program run.py, requirements for the Python program under requirements.txt and LICENSE and README.md files which are very common for repositories with public access. This subchapter focuses on the organizations/

directory, as it is responsible for creating and configuring testing organisations in GitHub – the `github-org/` subdirectory, GitLab – the `gitlab-org/` subdirectory and Bitbucket – the `bitbucket-org/` subdirectory. Each subdirectory contains configuration files aiming to create comparable organisations in each respective provider, according to the aforementioned architecture. Bitbucket organisation is configured using Python scripts, while GitHub and GitLab are setup using Terraform scripts. According to the official Terraform documentation, “*Terraform is an infrastructure as code tool that lets you build, change, and version infrastructure safely and efficiently*” [27]. All Terraform configurations are written in HashiCorp Configuration Language (HCL), which provides a declarative syntax for defining resources and their dependencies in a human-readable format. Reusable components are organised into modules, allowing the same infrastructure patterns – such as repository creation or permission assignments – to be applied consistently across providers without code duplication. Terraform standard workflow consist of initiating state file, which monitors active resource configuration and compares it with the current HCL configurations, then planning changes and applying them. Subdirectories `github-org/` and `gitlab-org/` are called modules in the Terraform nomenclature. A typical Terraform module contains of `.tf` configuration files, a `terraform.tf` file which configures Terraform providers used within the module and the state file (backend) location and a `variables.tf` file, which defines variables to be defined for the module. Such a module can be later reused, which simplifies configuration when dealing with larger Terraform projects. Terraform providers implement the resource types referenced in `.tf` configuration files and act as an intermediary between Terraform Core and the provider’s, translating configuration changes into operations on external resources. GitHub resources are defined in the `integrations/github` [28] provider, while GitLab resources are defined in the `gitlabhq/gitlab` [29] provider. As far as the Bitbucket is concerned, the reason this provider is not configured using Terraform is that Bitbucket does not have any stable, regularly updated single provider and for this reason configuring resources in Bitbucket is far more convenient using Bitbucket directly. In GitHub repositories can be grouped in the organisations, while in GitLab they can be grouped in the groups. In Terraform modules from the `organizations/` directory it is configured in the `organization.tf` file for the GitHub and the `group.tf` for the GitLab. GitHub organisation is created using the `github_organization_settings` Terraform resource, which has multiple options to define enabled or disabled, while GitLab group is not directly created by Terraform, but is rather read from GitLab to be used as a reference for other GitLab resources. While GitLab’s Terraform provider has a `gitlab_group` resource it is clearly stated that “*On GitLab.com, you cannot use the `gitlab_group` resource to create a top-level group. Instead, you must create a group in the UI, then import the group into your Terraform configuration.*” [30]. Therefore, as this thesis tests provider-hosted Git hosting (SaaS), GitLab group had to be created manually and then it can be used in Terraform configurations using the `gitlab_group` data resource.

```
1 resource "github_repository" "small-project" {
2   name           = var.small_project.name
3   description    = var.small_project.description
4   auto_init      = true
5   delete_branch_on_merge = true
6   visibility     = "private"
7
8   template {
9     owner      = "Krzychu-Z"
10    repository = "MODBUS-CRC16-Golang"
11    include_all_branches = true
12  }
13 }
14
15 resource "github_repository" "large-project" {
16   name           = var.large_project.name
17   description    = var.large_project.description
18   auto_init      = true
19   delete_branch_on_merge = true
20   visibility     = "private"
21
22   template {
23     owner      = "Krzychu-Z"
24     repository = "rust-compiler"
25     include_all_branches = false
26   }
27 }
28
29 resource "github_repository" "pipelines-repository" {
30   name           = var.pipelines_repository.name
31   description    = var.pipelines_repository.description
32   auto_init      = true
33   delete_branch_on_merge = true
34   visibility     = "private"
35 }
```

Listing 4. repo.tf file contents for GitHub

```
1 resource "gitlab_project" "small_project" {
2   name           = var.small_project.name
3   description    = var.small_project.description
4   namespace_id  = data.gitlab_group.parent.id
5   visibility_level = "private"
6   remove_source_branch_after_merge = true
7   default_branch = "master"
8   import_url     = "https://github.com/Krzychu-Z/MODBUS-CRC16-Golang.git"
9 }
10
11 resource "gitlab_project" "large_project" {
```

```
12     name                = var.large_project.name
13     description          = var.large_project.description
14     namespace_id         = data.gitlab_group.parent.id
15     visibility_level      = "private"
16     remove_source_branch_after_merge = true
17     default_branch       = "master"
18     import_url            = "https://github.com/Krzychu-Z/rust-compiler.git"
19 }
20
21 resource "gitlab_project" "pipelines_repo" {
22     name                = var.pipelines_repository.name
23     description          = var.pipelines_repository.description
24     namespace_id         = data.gitlab_group.parent.id
25     visibility_level      = "private"
26     initialize_with_readme = true
27     remove_source_branch_after_merge = true
28     default_branch       = "master"
29 }
```

Listing 5. repo.tf file contents for GitLab

Next in the repos.tf files for both GitHub and GitLab all three repositories are created, which is shown in Listings 4 and 5. The configurations declaratively define three private repositories – a small project, a large project, and a pipeline repository – each with custom descriptions, visibility set to private, ensuring they mirror realistic commercial setups with varying code sizes for performance benchmarking. A small project in each provider is cloned from a template GitHub repository <https://github.com/Krzychu-Z/MODBUS-CRC16-Golang.git>, which is a project that contains a Golang implementation of the fast MODBUS CRC 16-bit error-detecting algorithm. A large project on the other hand is cloned from a personal GitHub public template of the Rust programming language compiler <https://github.com/Krzychu-Z/rust-compiler>. Some resource parameters are set in both cases using variables. Variables in Terraform are a way to parameterize configuration files, enabling dynamic and reusable infrastructure definitions by substituting concrete values at runtime without altering the core logic. Input variables declared in variables.tf – such as repository names, descriptions, and parent group paths – accept external values through .tfvars files, command line flags, or environment variables, allowing the same IaC templates to provision identical environments across GitHub and GitLab. Variables in this example can be automatically read by Terraform from a terraform.auto.vars file. For GitHub and GitLab following variables must be defined: small_project, large_project and pipelines_repository Terraform objects containing name and description fields, email variable for both providers and organization name variable for GitHub and parent_group_path variable for GitLab. After the variables are set in the modules, each one can be applied in a way presented in the Listing 6 The only required action in this example, in order to authenticate to a Git hosting provider, is to export an appropriate Personal Access Token (PAT).

In github-org/ and gitlab-org/ in order to deploy both organisations, Terraform workflow presented in the Listing 6 is enough.

```
1 export PAT_TOKEN=*****
2 terraform init
3 terraform plan
4 terraform apply
```

Listing 6. Terraform workflow for applying GitHub and GitLab organisation/group resources

The Bitbucket provider is set up in a different way than GitHub and GitLab, because of the complications with using Terraform mentioned earlier. Therefore, this organisation is created using Python module which directly performs actions on the Bitbucket's . Python module is structured as a proper package within the bitbucket-org/ directory, containing `__init__.py` to enable imports, `bitbucket_config.py` as a central place for module variable definitions, `repo_functions.py` housing reusable functions for idempotent repository/workspace creation, and `main.py` which is a module executor file. The `requirements.txt` file is used to create Python lightweight virtual environment [31], which allows to install required dependencies in an isolated, reproducible manner without affecting the system-wide Python installation. The required dependencies for this module are requests in version from 2.31.0 to 3 – excluding this version and atlassian-python-api in version 4.0.7 and above. For the sake of clarity, the `main.py` file is described using pseudocode based on Python syntax. This approach avoids Python-specific technicalities and instead focuses on the repository and organization creation algorithm implemented in the file.

```
1 import os
2
3 API = "https://api.bitbucket.org/2.0"
4
5 CONFIG = {
6     'WORKSPACE_SLUG': "masters-thesis-workspace",
7     'PROJECT_KEY': "MTP",
8     'PROJECT_NAME': "masters-thesis-project",
9     'PROJECT_DESC': "Masters thesis project (created via atlassian-python-api)",
10    'IS_PRIVATE': True,
11    'CLIENT_ID': os.getenv("BB_CLIENT_ID"),
12    'CLIENT_SECRET': os.getenv("BB_CLIENT_SECRET"),
13    'USER_ACCOUNT_ID': os.getenv("BB_USER_ACCOUNT_ID")
14 }
15
16 REPOS = [
17     {
18         "name": "small-project",
19         "description": "Imported from GitHub template",
20         "import_url": "https://github.com/Krzychu-Z/MODBUS-CRC16-Golang.git"
```

```
21     },
22     {
23         "name": "large-project",
24         "description": "Imported from GitHub template",
25         "import_url": "https://github.com/Krzyschu-Z/rust-compiler.git"
26     },
27     {
28         "name": "pipelines-repository",
29         "description": "Empty repo initialized with README",
30         "import_url": None
31     }
32 ]
```

Listing 7. bitbucket_config.py file contents

The bitbucket_config.py file presented in the Listing 7 is a file that contains Bitbucket module variables, namely API, CONFIG and REPOS. The API variable is a Python string containing URL address of the Bitbucket's interface - <https://api.bitbucket.org/2.0>. The CONFIG variable is a Python dictionary (i.e., a dict) that stores configuration parameters as key-value pairs, including Bitbucket-specific details such as the workspace slug, which uniquely identifies the workspace in Bitbucket URLs and API calls, project details and Bitbucket credentials, which are extracted from the shell environment variables using the `os.getenv()` function. This module requires specific type of credentials known as *OAuth* keys and authenticate with Bitbucket's using the *OAuth 2.0* protocol, which is the industry-standard protocol for authorization [32]. This requires generating client identification key and client secret key for *OAuth* consumer, for which instruction is available under this link: <https://support.atlassian.com/bitbucket-cloud/docs/use-oauth-on-bitbucket-cloud/>. These credentials are later used to generate temporary *OAuth 2.0* access token, which is actually used when communicating with the Bitbucket's API. The last required variable is the REPOS Python list, which contains dicts with name, description and import_url keys, for each repository that needs to be created in Bitbucket.

```
1 def main():
2     Obtain OAuth2 token using bbconfig.CONFIG['CLIENT_ID'] and
3     ↳ bbconfig.CONFIG['CLIENT_SECRET']
4     Initialize Bitbucket Cloud client with OAuth2 token
5
6     If project bbconfig.CONFIG['PROJECT_KEY'] does not exist:
7         Create it in the specified workspace (bbconfig.CONFIG['WORKSPACE_SLUG'])
8
9     For repository in bbconfig.REPOS:
10         Create repository of key 'name' in the project
11
12         If 'import_url' is defined for this repository:
13             Perform mirror import of the code base from the specified URL
```



```

14         Else:
15             Initialise empty repository with only README.md file
16
17             Ensure user bbconfig.CONFIG["USER_ACCOUNT_ID"] has write access to this repository.

```

Listing 8. Pseudocode algorithm from the main.py file

The algorithm of the main.py file is described using pseudocode in the Listing 8. First, an OAuth2 access token is obtained using CLIENT_ID and CLIENT_SECRET from the bbconfig.CONFIG dictionary, and then it is used to initialise a Bitbucket Cloud client responsible for performing API operations. Next, the algorithm verifies whether the target project identified by PROJECT_KEY already exists and, if not, creates it within the workspace specified by WORKSPACE_SLUG, which ensures that all repositories are grouped consistently under a single namespace. Afterwards, for each entry in the bbconfig.REPOS list the script creates a repository with the configured name and, depending on whether the optional import_url field is provided, either performs a mirror import from the given URL or initialises an empty repository containing only a README.md file. Finally, the procedure ensures that the account referenced by USER_ACCOUNT_ID has write access to the newly created repository. Deployment of the Bitbucket resources using this Python module must follow steps defined in the Listing 9. It is very crucial that the steps below are executed within the bitbucket-org/ directory. First, a local Python virtual environment must be created and activated using the first two shell commands from the Listing. Then, the required dependencies are installed in this isolated environment using the third command and OAuth keys must be exported to the shell's environment for proper authentication. After successfully completing all aforementioned steps, Bitbucket infrastructure can be deployed by running the main.py file from the module.

```

1 python -m venv <venv>
2 source <venv>/bin/activate
3 pip install -r requirements.txt
4 export BB_CLIENT_ID=*****
5 export BB_CLIENT_SECRET=*****
6 python main.py

```

Listing 9. Bitbucket resources deployment workflow

3.2 Overview of CI/CD pipeline definitions

The testing environment created in the previous subchapter is used for running test pipelines and collecting the timestamps required for further analysis. All three vendors evaluated in this thesis are tested using two pipeline execution models, each applied to both a small and a large code base,

which results in four distinct experimental scenarios per vendor. This structure ensures that the measurements remain comparable across platforms and workload sizes. Two main execution models in this thesis are running pipelines in provider-hosted runners and running pipelines in self-hosted runner environment. This pipeline division does not impact the actual workflow that is defined inside, in order to make it possible to compare capabilities of provider-hosted runners to self-hosted runners. However, the pipelines for the small and large repositories have different structures, because they serve different purposes. Pipelines in small repositories follow typical *CI/CT/CD* workflow, that is clearly visible in each section of any such pipeline. Pipelines in large repositories focus mostly on extensive testing and delivering results.

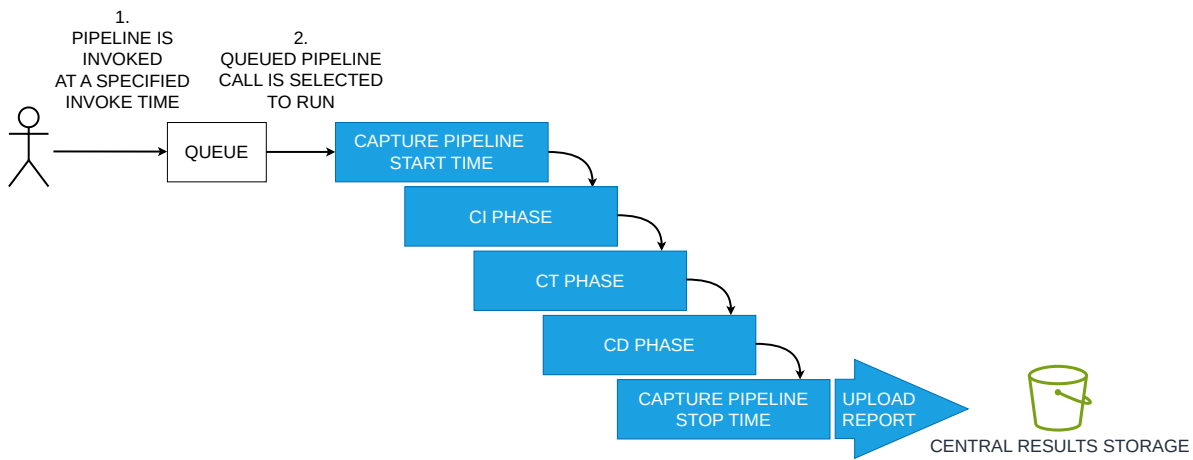


Figure 11. General simple pipeline workflow

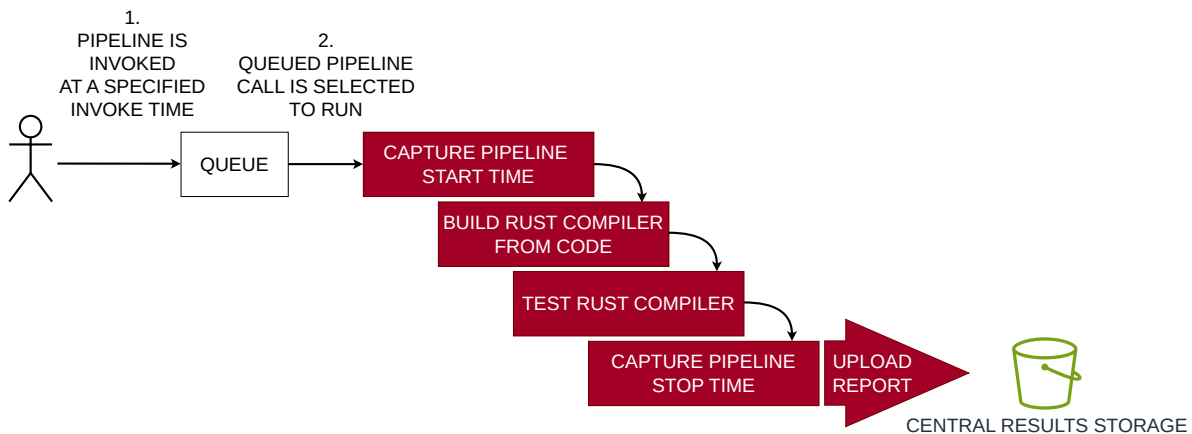


Figure 12. General Rust pipeline workflow

The Figure 11 showcases general workflow of small repository pipelines across each Git hosting provider, which are called *simple* in terms of actual CI/CD definitions. Once pipeline is invoked, preferably using code oriented solutions, this request is queued in pipeline queue before new pipeline

run is created. Each simple pipeline run consists of 6 phases, namely capture pipeline start time, CI phase, CT phase, CD phase, capture pipeline stop time and generate and upload the results to a central result storage. For each provider the actual pipeline configurations look differently, according to possibilities of each provider, however they still achieve the same functionality. The Figure 12 describes general workflow of large repository pipelines. The only difference in this case, as opposed to small repository workflows, is the details regarding what the pipeline actually does. The general purpose of pipelines in large repositories is to record different timestamps for the process of building and testing a Rust language compiler.

GitHub supports reusable workflows, meaning that a separate repository can store complete pipelines or reusable pipeline components that other repositories can reference. This mechanism simplifies debugging and maintenance, because updates can be applied in one place and propagated consistently. As a result, newly created repositories within the organisation can keep their workflow definitions lightweight. As far as GitHub workflows are concerned, source repositories – where the pipeline is executed – make use of this mechanism by referring to reusable workflows from the pipelines-repository template repository.

```

1  ---
2  name: Simple performance pipeline call
3  on:
4    workflow_dispatch:
5      inputs:
6        pipeline-call-ts:
7          description: 'Timestamp of the pipeline call.'
8          required: true
9
10
11 jobs:
12   call-workflow:
13     name: Call simple performance CI/CD pipeline
14     uses: masters-thesis-org/pipelines-repository/.github/workflows/simple-template.yml@main
15     with:
16       pipeline-call-ts: ${ inputs.pipeline-call-ts }
17     secrets:
18       aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
19       aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
20       aws-region: ${ secrets.AWS_REGION }
21       s3-bucket-name: ${ secrets.S3_BUCKET_NAME }

```

Listing 10. GitHub small repository source workflow definition

The workflow definition present in the Listing 10 calls reusable workflow `simple-template.yml` using the `uses:` object – for more details refer to the full file in management repository’s GitHub <https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/github/pipeline-repo>

sitory/simple-template.yml – from the pipelines-repository. It also requires pipeline call timestamp as an input variable and passes further variables and secrets to the reusable workflow, which are required for proper report generating. The simple-template.yml reusable workflow also reuses other two reusable workflows, namely the ci__simple__small-project.yml – see GitHub https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/github/pipeline-repository/ci__simple__small-project.yml – and the cd__simple__small-project.yml – see GitHub https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/github/pipeline-repository/cd__simple__small-project.yml – which is made so as to reduce the length of the simple-template.yml. In GitLab pipelines can also reuse pipelines from other sources. The main difference between GitLab and GitHub is that, in GitHub, reusable workflows must be placed in the .github/workflows/ directory and this directory cannot contain subdirectories, so all workflow YAML files are stored directly in that location. In GitLab, YAML pipeline definitions do not have this restriction and can be organised in nested directory structures.

```
1 variables:
2   PIPELINE_CALL_TS: ""
3
4 stages: [triggers]
5
6 simple_perf:
7   stage: triggers
8   trigger:
9     include:
10    - project: 'masters-thesis-group/pipelines-repository'
11      ref: master
12      file: '.gitlab/ci/simple-template.yml'
13    strategy: depend
14    forward:
15      pipeline_variables: true
16  rules:
17    - if: '$PIPELINE_TYPE == "simple-perf"'
18
19 k8s_perf:
20   stage: triggers
21   trigger:
22     include:
23    - project: 'masters-thesis-group/pipelines-repository'
24      ref: master
25      file: '.gitlab/ci/k8s-simple-template.yml'
26    strategy: depend
27    forward:
28      pipeline_variables: true
29  rules:
30    - if: '$PIPELINE_TYPE == "k8s-perf"'
```

Listing 11. GitLab small repository source workflow definition

The workflow definition presented in Listing 11 defines the entire GitLab pipeline for the small repository in the `.gitlab-ci.yml` file, where two alternative trigger jobs are declared, namely `simple_perf` and `k8s_perf`. The `simple_perf` is just a provider-hosted version of the small repository pipeline, while the `k8s_perf` is a self-hosted version of the same pipeline. Each of these jobs uses the `trigger:include` mechanism to call a reusable pipeline definition stored in the `pipelines-repository` project, selecting either `simple-template.yml` <https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/gitlab/pipeline-repository/.gitlab/ci/simple-template.yml> or `k8s-simple-template.yml` <https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/gitlab/pipeline-repository/.gitlab/ci/k8s-simple-template.yml> from a specific branch (`ref: master`), and executes it with `strategy: depend` so that the parent pipeline waits for the triggered pipeline to finish and propagates its status. Both `simple-template.yml` and `k8s-simple-template.yml` reuse other YAML files, which are also available in the repositories mentioned earlier. Since GitLab requires that repository pipelines are defined in a single `.gitlab-ci.yml` entry file, the selection between the two execution models is implemented using an additional input variable `PIPELINE_TYPE` and `rules` clauses, so that only the matching trigger job is executed for a given run. Furthermore, the variable `PIPELINE_CALL_TS` is declared and forwarded to the triggered pipeline (`forward: pipeline_variables: true`) to preserve the pipeline call timestamp, while other variables required for report generation can be propagated in the same way.

Configuring CI/CD pipelines in Bitbucket Free works very differently as opposed to GitHub and GitLab. As stated in the Bitbucket documentation, “*Pipelines configuration sharing is a Bitbucket Cloud Premium feature*” [33]. Since this thesis analyses by default free versions of each provider, further assumptions state that Bitbucket does not support pipeline sharing and that all definitions must be made in a single `bitbucket-pipelines.yml` file. Bitbucket pipeline definitions are available in `bitbucket-pipelines.yml` (<https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/bitbucket/small-repo/bitbucket-pipelines.yml>) – for a small repository and in `bitbucket-pipelines.yml` (<https://github.com/Krzychu-Z/ci-cd-comparison/blob/main/pipelines/bitbucket/large-repo/bitbucket-pipelines.yml>) for a large repository. Despite pipeline sharing is not possible in this version of Bitbucket, configurations presented in the aforementioned appendices relay the main logic of some individual steps to Bash scripts placed in the `scripts/` directory, in order to make the YAML configuration files more readable. To make it even more readable this pipeline configuration makes use of the YAML anchors, described in the section 2.3 of the thesis. Both Bitbucket pipelines achieve the same functionality as the GitHub and GitLab ones. At this point it is also vital to explain why there are no self-hosted pipeline definitions in appendices. The reason for this is because the only difference between the provider-hosted pipelines and the self-hosted pipelines in this thesis is one variable in each step or job for each provider. Other YAML objects and variables stay the same except for `runs-on`: variable for GitHub and Bitbucket, and GitLab tags: list with specified tag `k8s` which is also attached to self-hosted runner set.

3.3 Self-hosted runner scale set configurations

This thesis compares the capabilities of CI/CD pipelines in a holistic manner. In particular, it evaluates provider-hosted pipelines against self-hosted pipelines for each platform. This subchapter describes how the local environment is configured to host runner sets for GitHub, GitLab, and Bitbucket. The overall architecture of the solution is to create a full Kubernetes cluster and register runner scale set for each of the providers. *“Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications”* [34]. From the perspective of this thesis containers and Kubernetes can be treated like a “blackbox”, meaning the only use of this technology in this thesis is to host multiple self-hosted runners efficiently without the need to dissect the exact components or architecture of this solution. Therefore, this chapter only briefly showcases how this environment is set up. The easiest way to create a Kubernetes cluster is to deploy it in a cloud environment using a managed Kubernetes service provided by the cloud operator, because it abstracts most of the operational complexity and allows the cluster to be created and scaled with minimal manual configuration. Kubernetes cluster for this thesis is created in Amazon Web Services (AWS) cloud provider using the Elastic Kubernetes Service (EKS) managed service for operating on Kubernetes clusters. The creation process for this Kubernetes cluster is managed using IaC approach. In particular, Terraform configuration is placed in the `environment/` directory of the management repository under `eks.tf` file. This terraform file essentially calls two Terraform community modules, namely the `terraform-aws-modules/eks/aws` [35] and `terraform-aws-modules/vpc/aws` [36]. The last module configures networking resources for proper Kubernetes cluster setup. In Amazon EKS, the worker machines that actually run pods – in Kubernetes containers are managed inside a Kubernetes object called pod – are typically managed as node groups, which define a pool of compute instances attached to the cluster. Node groups allow the cluster capacity to be scaled and updated in a controlled manner, so that runner workloads can be adjusted to the required load. Compute instances in this case refer to virtual machines, which are available in various sizes (instance types) and pricing models. The node group used for the Kubernetes cluster in this thesis consists of two instances running in the Spot pricing model and using the `t3.large` instance type, which provides 2 vCPUs, 8 GiB of memory and up to 5 Gbps burst network bandwidth. Root volume disks have their size increased from standard 20 GiB to 40 GiB in order to provide enough space for multiple runners from each Git hosting provider to run simultaneously. The whole process of setting up a fully working EKS cluster in AWS using Terraform configuration defined in the management repository is designed in such a way, that it is created from scratch by running `terraform apply` command. This allows to fully automate Kubernetes cluster provisioning, as well as to destroy the cluster and all related resources by running `terraform destroy -target module.vpc -target module.eks` command, which can greatly save costs when tests of CI/CD pipelines are not run.

```

1 helm install arc \
2   --namespace "${ARC_SYSTEM_NS}" \
3   --create-namespace \
4   oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set-controller
5
6 helm install "${INSTALLATION_NAME}" \
7   --namespace "${ARC_RUNNERS_NS}" \
8   --create-namespace \
9   --set githubConfigUrl="${GITHUB_CONFIG_URL}" \
10  --set githubConfigSecret.github_token="${GITHUB_PAT}" \
11  --set runnerGroup="masters-thesis-arc-group" \
12  oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set

```

Listing 12. GitHub ARC runner set installation using Helm charts

After the EKS cluster is successfully set up, runner scale sets are deployed for each provider. There are three Bash scripts in the `environment/` directory in the management repository, that perform this task, namely `github-arc.sh`, `gitlab-arc.sh` and `bitbucket-arc.sh`. In GitHub self-hosted Kubernetes runners implementation used in this thesis is called Actions Runner Controller (ARC) runner set and can be deployed in the cluster by running commands from the Listing 12. This listing presents two Helm chart being installed in the EKS cluster. Helm is a widely used Kubernetes package manager that simplifies deploying and maintaining complex applications by describing them as reusable *charts*. The first chart installs the ARC controller components in a dedicated system namespace, while the second chart provisions the runner scale set in a separate namespace and configures it with the GitHub URL, authentication token, and runner group. Helm is used here purely as a practical installation mechanism and remains beyond the scope of this thesis, meaning only the resulting runner infrastructure is relevant for the conducted experiments. More details on how to deploy GitHub ARC runner set can be found in the official GitHub documentation [37].

```

1 helm repo add gitlab https://charts.gitlab.io >/dev/null 2>&1 || true
2 helm repo update gitlab >/dev/null 2>&1 || true
3
4 helm upgrade --install "${GITLAB_RELEASE_NAME}" \
5   --namespace "${GITLAB_NAMESPACE}" \
6   --create-namespace \
7   --set gitlabUrl="${GITLAB_URL}" \
8   --set runnerRegistrationToken="${GITLAB_REGISTRATION_TOKEN}" \
9   --set rbac.create=true \
10  --set concurrent=12 \
11  --set runners.tags="{eks,k8s}" \
12  gitlab/gitlab-runner

```

Listing 13. GitLab ARC runner set installation using Helm charts

Listing 13 shows the Helm-based installation of GitLab Kubernetes runners using the official `gitlab/gitlab-runner` chart. First, the GitLab Helm repository is added and updated to ensure the latest chart version is available, and then the chart is installed (or upgraded) into a dedicated Kubernetes namespace using `helm upgrade --install`. During installation, the runner is configured with the target GitLab instance URL and registration token, `rbac.create` is enabled to create the required Kubernetes permissions, and parameters such as `concurrent` and `runners.tags` are set to control the maximum parallel job execution and to label the runners for later job selection. Further details on deploying and configuring GitLab Runner via the Helm chart are provided in the official GitLab documentation [38]. As far as Bitbucket Kubernetes runners are concerned, Atlassian does not publish any official Helm charts, which means there is no official automated way to install them. However, Bitbucket Support page published instructions [39] on how to do it manually using Kubernetes resources defined in a GitHub `runners-autoscaler` repository. Therefore, to automate the complete Kubernetes setup process for this thesis, the steps described in the documentation are consolidated and implemented in the `bitbucket-arc.sh` script. First, the script prepares the Bitbucket runner autoscaler deployment by retrieving the upstream autoscaler manifests and creating a small, local configuration overlay that describes the target workspace, runner group, labels, and basic scaling limits. Next, it injects the required Bitbucket authentication parameters into the Kubernetes resources so that the autoscaler is allowed to register and manage runners on behalf of the workspace. Finally, the prepared manifests are applied to the cluster, which enables runners to be provisioned and scaled automatically in response to queued pipeline workloads. Once all runner sets have been deployed in an EKS cluster, a list of pods presented in the Listing 14 should be running inside.

```

1 $ kubectl get pods -A
2 NAMESPACE                                NAME                                READY   STATUS
3 bitbucket-runners                        runner-ab521a8c-75c5-5f61-ae96-2055d70765e1-hcvdr  2/2
4 ↪ Running
5 bitbucket-runners                        runner-controller-6cbfc66fc-djppm             1/1
6 ↪ Running
7 bitbucket-runners                        runner-controller-cleaner-76cd6646c-cbvkh       1/1
8 ↪ Running
9 gitlab-runners                          gitlab-runners-9f47bc8b4-9mkm9                 1/1
10 ↪ Running
11 .
12 .
13 .
14 masters-thesis-arc-system               arc-gha-rs-controller-69dcbb7b6d-pc25r          1/1
15 ↪ Running
16 masters-thesis-arc-system               masters-thesis-arc-runner-set-5dfcc67d-listener  1/1
17 ↪ Running

```

Listing 14. Example showing all runner-related pods in Kubernetes cluster

3.4 Test results data pipeline

The last subsection of this chapter describes how data generated by pipeline runs is stored and analysed in this thesis. All pipelines upload their run results as JSONL files to a central results storage location. In this work, the central storage is implemented as an AWS Simple Storage Service (S3) bucket. AWS S3 is a managed object storage service in which data is stored as immutable objects identified by keys within a bucket, making it well suited for this work because it provides a simple, vendor-neutral drop-off point for artifacts produced by GitHub, GitLab, and Bitbucket pipelines. S3 bucket also offers easy integration with later processing steps, so the experimental results can be analysed without coupling the evaluation to any single CI/CD platform's internal artifact system. As each pipeline uploads a JSONL file to an S3 bucket in its last step, it uploads it to a prefix specified by a date, which is useful information for further statistical analysis but also useful for parsing the data by the test results pipeline.

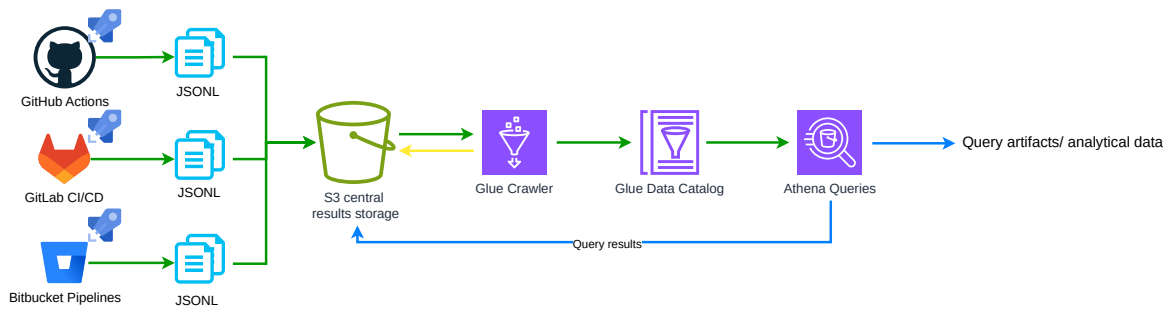


Figure 13. Test result data pipeline simple diagram

While JSONL files stored in an S3 bucket are enough to be downloaded, merged and analysed directly in a spreadsheet, AWS provides a way to perform simple transformations and analysis on data stored in S3 buckets without the need to download and transform all the data manually. For this purpose Amazon introduced AWS Athena service, which enables SQL queries over files stored in S3 and can be used to filter, aggregate, and prepare the experimental results in a lightweight manner before further processing. Athena is serverless, meaning it does not require provisioning or managing any database infrastructure, and queries can be executed on demand against the current contents of the bucket. In this thesis, this makes it convenient to validate collected measurements, compute summary statistics, and extract subsets of runs for further analysis while keeping the raw results in a single central location. Athena in order to be able to analyse data in an S3 bucket, requires to have some form of a database that holds all the data stored in S3 bucket usually under various prefixes – which are S3 version of directories for data. This database is created and maintained using AWS Glue service, in particular Glue Data Catalog, which contains databases and tables required by Athena. Updating such a database, on the other hand, is a responsibility of Glue Crawler, which has instructions where exactly in S3 bucket the data resides and in what format. The Figure 13 shows exactly that workflow. Each platform uploads its pipeline-run results as JSONL artifacts to

a single, central S3 results bucket, which acts as a neutral storage layer shared across all vendors. On top of this raw storage, an AWS Glue Crawler is used to automatically discover the structure of the uploaded JSONL files and register their schemas in the Glue Data Catalog, so that the dataset becomes queryable without manual table definitions. Finally, AWS Athena runs SQL queries directly against the data stored in S3, through the catalog metadata, producing query results and derived analytical artifacts that can be exported or further processed in subsequent analysis steps.

```
1 SELECT
2     avg(
3         date_diff(
4             'millisecond',
5             date_parse(pipeline_start_time, '%H:%i:%s.%f'),
6             date_parse(pipeline_stop_time, '%H:%i:%s.%f')
7         ) / 1000.0
8     ) AS avg_diff_seconds
9 FROM
10     "masters-results"."run_date_2025_12_06"
11 WHERE
12     "$path" LIKE '%gitlab%'
```

Listing 15. Example AWS Athena SQL query

Listing 15 shows an example SQL query performed on real data from tests executed for this thesis. The query computes the average pipeline execution time by taking the difference between `pipeline_start_time` and `pipeline_stop_time`: timestamps are first parsed with `date_parse`, then the duration is calculated using `date_diff` in milliseconds and converted to seconds by dividing by 1000. The `WHERE` clause filters the dataset by vendor, which makes it possible to obtain vendor-specific metrics for a selected run date table (here: `run_date_2025_12_06`). Considering the fact that even more filtering can be performed, almost all statistical parameters for this thesis are calculated in that way. Such SQL queries can also return columns, rows or any other kind of a number list, which is beneficial for drawing charts. The charts for this thesis are drawn using Microsoft Excel and the data extracted from Athena SQL queries.

Chapter 4

CI/CD pipeline evaluation

This chapter presents the results of the experiments and analyses the behaviour of CI/CD pipelines on the three examined platforms. Its role is to discuss in detail the results of running various tests on the architecture described and created in the previous chapters.

First, the chapter describes the ease-of-use of CI/CD workloads in each of the selected providers. It compares how GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines express pipeline structure, how runners are configured, and how typical maintenance tasks are performed. The results in this part are based on functionalities of the platforms and on the experience gained while preparing and running the experiments.

In the final part, the chapter describes tests of CI/CD pipelines in terms of numerically measurable performance. It explains the test scenarios and then reports metrics such as pipeline duration, pipeline latency and expected wait in queue time. The analysis differentiates pipeline performance based on whether it is run on provider infrastructure or self-hosted. The analysis follows the notation introduced in the theoretical chapter and, where possible, interprets the observed behaviour in terms of $M/G/1$, $G/G/1$ and $G/G/c$ queueing models, answering the question if any runner system can be modelled by the queueing theory. The results are analysed statistically and presented using tables and plots for different pipeline types and sizes of the source code base. The results obtained here are later used in the concluding chapter to formulate recommendations for choosing between provider-hosted and self-hosted CI/CD configurations.

4.1 User-centric evaluation of CI/CD pipelines

Following subsection presents a subjective comparison of the configuration and management experience between GitHub Actions, GitLab CI/CD and Bitbucket Pipelines. This part gathers holistically experience from defining, debugging and managing pipelines, taking into account any context related with the criteria against which pipelines are evaluated. CI/CD pipeline evaluation would not be complete, if it did not discuss the way features of CI/CD pipelines for each provider are exposed to the user and how they affect day-to-day workflow, such as the effort required to introduce new pipeline variants, reuse existing definitions, and troubleshoot failures in a maintainable and reproducible manner. This subsection investigates the thesis question mentioned in the Introduction chapter, namely how the platforms differ in configuration model and ease of use for typical project workflows?

Simple performance pipeline call - self-hosted version #73

Summary

All jobs

- Call simple performance CI/CD pipeline
- Capture Start Time
- static-analysis
- ci-tests**
- ct-tests
- Build binaries (ubuntu-latest)
- Build binaries (windows-latest)
- Build binaries (macos-latest)
- Capture Stop Time
- Generate Performance Report

Run details

Usage

Workflow file

Annotations
3 warnings

Call simple performance CI/CD pipeline / Continuous Integration and Continuous Testing / ci-tests
succeeded on Dec 12, 2025 in 21s

Search logs

Step	Duration
Set up job	2s
Checkout code	1s
Setup Go	1s
Run unit tests	13s
Measure build time (start)	0s
Build binary	0s
Simple functional test (CRC check)	0s
Measure build time (stop)	0s

```

1 Run chmod +x ./modbusCRC16
2 chmod +x ./modbusCRC16
3 ./modbusCRC16 011000110003061AC4BAD0 1 > out.txt
4 grep "0x677F" out.txt
5 shell: /usr/bin/bash -e {0}
6 Obliczona suma kontrolna CRC: 0x677F

```

Figure 14. GitHub example pipeline logs

4.1. User-centric evaluation of CI/CD pipelines

masters-thesis-group / small-project / Jobs / #12703588875

Log timestamps in UTC. Search visible log output

```
22 19:30:44 using docker image sha256:2fceb9c7a03/code112034eed0f0a10313ced1c7f7ac00023/
8642c2c7ca84 for go lang:1.22 with digest go lang@sha256:1cf6c45ba39db9fd6db1692
2041d074a63c935556a05c5ccb62d181034df7f02 ...

23 19:30:45 $ go test -v ./src/...
24 19:30:56 === RUN TestCalculateCRCFast
25 19:30:56 --- PASS: TestCalculateCRCFast (0.00s)
26 19:30:56 PASS
27 19:30:56 ok      github.com/Krzychu-Z/MODBUS-CRC16-Go lang/src      0.002s
28 19:30:56 $ echo "CI_START_TIME=$(date +%T.%N)" > ci_times.env
29 19:30:56 $ go build -o modbusCRC16 ./src/modbusCRC16.go
30 19:30:56 $ chmod +x ./modbusCRC16
31 19:30:56 $ ./modbusCRC16 011000110003061AC4BAD0 1 > out.txt
32 19:30:56 $ grep "0x677F" out.txt
33 19:30:56 Obliczona suma kontrolna CRC: 0x677F
34 19:30:56 $ echo "CI_STOP_TIME=$(date +%T.%N)" >> ci_times.env
35 19:30:56 Uploading artifacts for successful job
36 19:30:57 Uploading artifacts...
37 19:30:57 modbusCRC16: found 1 matching artifact files and directories
38 19:30:57 out.txt: found 1 matching artifact files and directories
39 19:30:58 Uploading artifacts as "archive" to coordinator... 201 Created correlation_id
=6949ef7c55e46ceaaf9ab1ca21d055b id=12703588875 responseStatus=201 Created to
ken=eyJraWQiO
40 19:30:58 Uploading artifacts...
41 19:30:58 ci_times.env: found 1 matching artifact files and directories
42 19:30:59 Uploading artifacts as "dotenv" to coordinator... 201 Created correlation_id=
685aeb4b9e5e4a56a32578e1b229a43a id=12703588875 responseStatus=201 Created tok
en=eyJraWQiO
43 19:30:59 Cleaning up project directory and file based variables
44 19:30:59 Job succeeded
```

Duration: 42 seconds
Finished: 1 week ago
Queued: 0 seconds
Timeout: 1h (from project) ?
Runner: #32976645 (YKxHNyex) 6-green.saas-linux-small-amd64.runners-manager.gitlab.com/default
Source: Parent Pipeline

Job artifacts ?
These artifacts are the latest. They will not be deleted (even if expired) until newer artifacts are available.

Keep Download Browse

Commit 5ab5ed78 ?
Fixing self-hosted in gitlab

Pipeline #2261013053 ? Passed for master ?

ci

Related jobs
→ ? ci_tests

(a) GitLab example pipeline logs

masters-thesis-workspace / ... / Pipelines

#640 Rerun

634ef1f Fixed typo in pipeline
master
custom: simple-performance-k8s
Learn more about reports
7min 13 sec December 13, 2025

Pipeline

- Capture pipeline start time 5s
- Super-linter 15s
- CI tests (unit + build + functional) 20s
- CT tests (long-running performance) 5m 23s
- Build linux amd64 tarball 35s
- Build windows amd64 tarball 14s
- Build macOS amd64 tarball 15s

Build Metrics NEW

Runner

Build setup

```
bash scripts/ci_tests.sh
1 + bash scripts/ci_tests.sh
2 go version go1.22.12 linux/amd64
3 === RUN TestCalculateCRCFast
4 --- PASS: TestCalculateCRCFast (0.00s)
5 PASS
6 ok      github.com/Krzychu-Z/MODBUS-CRC16-Go lang/src      0.004s
7 Obliczona suma kontrolna CRC: 0x677F
8
```

Build teardown

(b) Bitbucket example pipeline logs

Figure 15. Example pipeline logs

The first category that is evaluated in this subsection is pipeline debugging capabilities for each provider. From the perspective of day-to-day development, the ergonomics of debugging pipelines differs noticeably across the three providers. GitHub Actions, as shown in the Figure 14, provides the clearest separation of logs by job and step, so the developer can quickly locate the failing phase and re-run only the relevant part, which makes iterative debugging efficient even for longer workflows. GitLab's logs window is presented in the Subfigure 15a. While rich in metadata, it tends to concentrate most information into a single job log view, keeping everything in one continuous stream. This makes the output harder to scan, especially when the job mixes setup, build, tests, and artifact uploads. It is also worth noting, that the logs window in Subfigure 15a has its background purposefully coloured white by manipulating HTML values, which purpose is to improve text readability in this thesis. By default background colour of the logs window in GitLab CI/CD is black, which is better suited for reading large streams of logs. Bitbucket Pipelines, which example logs are presented in the Subfigure 15b, sits somewhere in between, but its weakest point is the clarity of platform-side error reporting: when the failure is caused by runner or infrastructure constraints (e.g., ephemeral disk space issues on the execution environment), the surfaced messages can be ambiguous and therefore time-consuming to diagnose. Bitbucket Pipelines logs window is manually colour-inverted so the logs are shown on a light background rather than a black one. Bitbucket Pipelines logs, present in the Bitbucket UI, are placed on a black background for the same reasons GitLab CI/CD shows it using similar colours.

Next important issue regards pipeline reusability and automation capabilities of each solution. It is a common thing for many larger organisations to define multiple pipelines, where each pipeline performs different tasks and some of them might share some steps/stages. Trying to read or debug such pipelines ideally should be very straightforward and user-friendly. Furthermore, ideally running pipelines should be fully automated, meaning that no human intervention should be needed in this process, once appropriate configuration files are published. Each provider approaches these issues differently. As for GitHub, workflows can be reusable. Reusable workflows can be placed in any GitHub repository, but must be placed in `.github/workflows` directory in a flat hierarchy – meaning no subdirectories are allowed. This is perfect situation when there are few GitHub repositories and each has its own unique reusable workflows. However, when a team decides that they create central repository with pipeline templates, all of the templates are stored in `.github/workflows` which means this directory may grow in up to tens or even hundreds of YAML files. Therefore, it is very important, that if a team decides for such a central reusable workflows repository, each template file name is easily distinctive among all other files – especially when reusable workflows refer to other reusable workflows from the same directory. While allowing reusable workflows to refer other may be beneficial in reducing size of such a file, it may be harder to navigate and read the logic of the target version of a pipeline. In GitLab all pipelines must be defined by default in a `.gitlab-ci.yml` file. However, this file can use other YAML files, which means that this file can work similar to what is offered by the `.github/workflows` directory in GitHub, as well as it may also mean that only

repetitive steps are “exported” to other locations. GitLab does not limit users on where YAML files must reside, so when a team decides to create a central pipeline template repository, they can group all the files in a directory hierarchy, which greatly improves navigation between files. When combining this information with another information, that GitLab allows nested YAML files calling, it allows to keep extensive pipeline configurations in small and easy to read files. Last, but not least Bitbucket (Free) offers the least flexibility when it comes to defining pipelines. All pipeline definitions must be kept in a single file titled `bitbucket-pipelines.yml` and the Free version does not allow reusing pipelines from other locations. For that reason these files easily become very large and difficult to read, but it also may mean developers do not need to consider looking up other files regarding the pipeline. Two major improvements can be made to significantly increase readability of such files. The first one being the utilisation of the YAML anchors, described in the Section 2.3 and the second one is that Bash instructions for any step can be saved as a `.sh` file and referenced by a `script:` key.

As far as automation is concerned for each provider, in general pipelines of all providers are almost fully automatable with few caveats. Depending on the trigger type – which is actually true for all providers – GitHub Actions are fully automatable, unless such a workflow refers any reusable workflow from other repository, even if the repository belongs to the same user or is in the same GitHub organisation. Therefore, it is important to allow all actions and reusable workflows in the source repository settings, otherwise the pipeline fails with insufficient permissions error. On GitHub repository page such a setting can be found in the Settings tab under Actions/General option. In addition, GitHub restricts secrets and write permissions for workflows triggered from forked repositories under the `pull_request:` event, which may prevent fully unattended execution of jobs that require repository-scoped tokens or other sensitive variables. GitLab CI/CD pipelines offer the biggest level of automation across all providers. One practical caveat in this case, is that when GitLab pipelines are decomposed into child pipelines via `trigger/include`, variables and secrets are not always available by default and must be explicitly forwarded, otherwise the triggered pipeline may execute without the required context. Bitbucket also offers similar levels of automation. However, initially pipelines must be manually enabled in the repository settings and later pipelines of type `custom` must also be started manually – next pipeline triggers can be made using calls. Bitbucket also offers pipeline templates (starter configurations), that can generate an initial `bitbucket-pipelines.yml` and run automatically for common workflows, but they are designed for standard use cases and are not used in this thesis. Another limitation is that fully automated runs typically still depend on repository/workspace variables and secured credentials configured in Bitbucket settings (or via API), because secret material is not defined purely in the `bitbucket-pipelines.yml` file and missing variables often manifest as ambiguous runtime failures.

Provisioning and managing self-hosted infrastructure also differs among each provider. While simple self-hosted runners can easily be configured anywhere using wizards or simple documentation

instructions – it mostly works either as an operating system process or a single container, configuring runner sets in Kubernetes can be more complicated. Most organisations seeking self-hosted solution choose Kubernetes versions over standalone containers or processes, because configuring self-hosted runners this way allows for automated scalability, greater reliability of the system and other benefits. To properly install the GitHub ARC runner set, two Helm charts must be provisioned, namely the `gha-runner-scale-set-controller` chart, which deploys the controller components responsible for managing runner resources in the cluster, and the `gha-runner-scale-set` chart, which defines the actual runner scale set and its configuration, including the GitHub URL, authentication token, and runner group. This setup allows for almost automatic deployment of the runner set. GitLab offers even easier implementation of their runner set in Kubernetes, as the solution architecture is simpler and only one Helm chart must be installed, namely the `gitlab/gitlab-runner` one. Deployment of the Bitbucket runner set is the most challenging, as Bitbucket does not officially maintain a Helm chart that would provision all required Kubernetes resources, and therefore the recommended setup relies on downloading, editing, and applying raw Kubernetes object YAML manifests. This approach is more error-prone and harder to standardise across environments, because the operator has to manage low-level manifest details (namespaces, secrets, RBAC, deployments) and ensure that updates to individual components do not impact runner environment. At the same time, it can be advantageous for some advanced configurations, since working directly with Kubernetes resources makes it possible to fine-tune runner behaviour for specific use-cases beyond what a fixed Helm chart interface would typically expose.

The last aspect considered is the financial one. All three vendors limit the amount of time that jobs can run on provider-hosted runners in their free tiers, so pricing for additional CI/CD minutes has a direct impact on usability. To make the offerings comparable, this section normalizes prices to the cost of 1000 additional compute minutes per month on each platform. For GitHub, a personal or organization account on the Free plan receives 2000 GitHub Actions minutes per month for private repositories. Public repositories, on the other hand, can use Actions on standard runners without a minute quota. When the included minutes are exhausted, usage on private repositories can continue as long as a spending limit is configured, and excess minutes are billed per-minute according to the runner type. For the standard 2-core Linux runner, the current price is 0.006\$ per minute, which means roughly 6\$ per 1000 additional minutes per month. This pay-as-you-go model is particularly attractive for small teams or academic projects that only occasionally exceed the free minute allowance, because they can buy additional capacity without committing to a higher subscription tier. If the spending limit remains at 0\$, new workflows on private repositories are blocked once the 2000 minute quota is reached. For GitLab, the Free tier provides 400 compute minutes per month for the whole top-level group. Unlike GitHub, GitLab sells explicit compute-minute packs that can be attached to the Free plan: additional minutes are available at 10\$ for 1000 compute minutes per month. When the monthly quota is exhausted, further jobs on shared runners are blocked until more minutes are bought or the quota resets. This model offers better cost

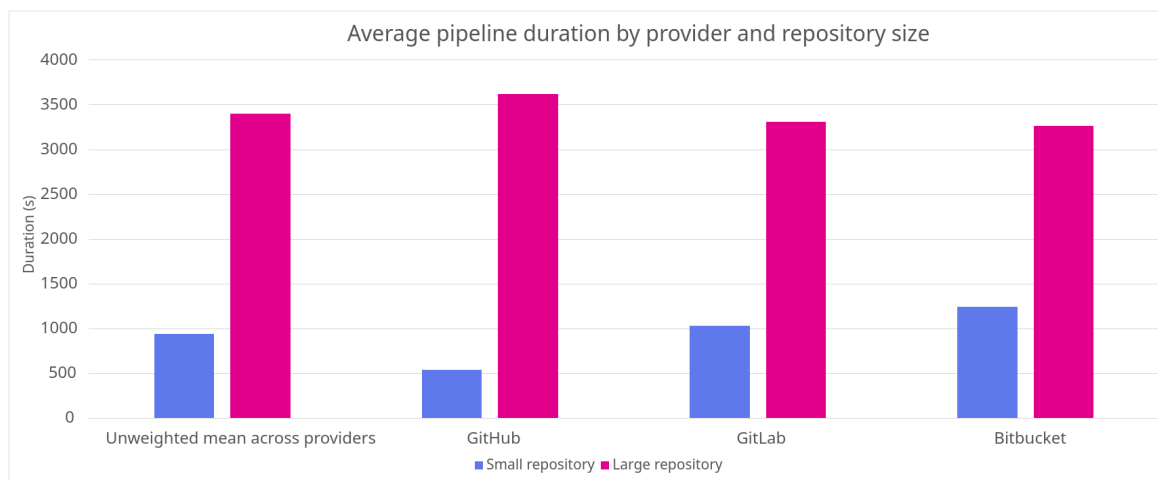
management, as teams purchase CI minutes in fixed blocks and can cap their maximum monthly spend in advance. For Bitbucket, the Free plan includes only 50 build minutes per month, which is the least of all considered providers, while the Standard and Premium plans include 2500 and 3500 minutes respectively. Bitbucket also offers build-minute add-ons: on any plan, including Free, teams can buy 1000 additional build minutes for 10\$ per month, which is directly comparable to GitLab's price point. After the included minutes are consumed, Bitbucket stops starting new pipelines until either the next billing cycle or additional minutes are purchased. From a cost perspective this means that, although Bitbucket offers the smallest free allowance, its effective price for extra capacity is similar to GitLab's and more predictable than GitHub's per-minute billing. This subchapter is summarised in Table 2 cumulating all user-centric aspects discussed here.

Table 2. User-centric aspects related to running CI/CD pipelines in GitHub, GitLab, and Bitbucket

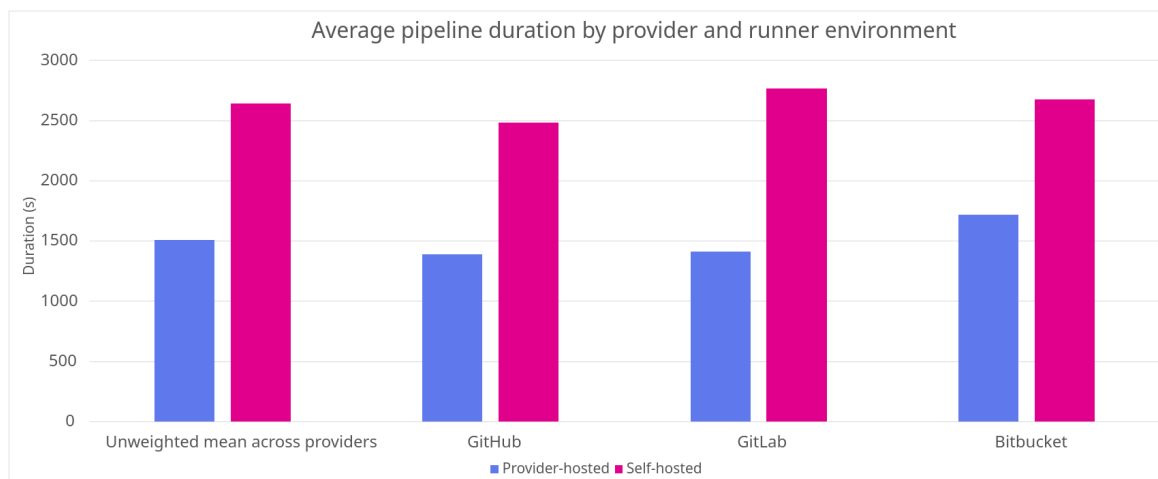
Aspect	GitHub	GitLab	Bitbucket
Pipeline debugging/log display	Logs separated by steps and jobs, easiest to read	All logs from a single job are placed together, challenging to read	Logs clearly separated by steps and jobs, can produce ambiguous messages
Pipeline reusability	Pipeline nesting is unlimited, pipeline location is constrained to <code>.github/workflows/</code>	Pipeline nesting is unlimited, pipelines must be defined in <code>.gitlab-ci.yml</code> but all other YAML files can be placed anywhere	Pipeline reusability is present only in higher plans
Automation capabilities	Fully automatable, needs enabling cross-repo reusable workflows and forked-PR secrets	Highest automation, child pipelines require explicit forwarding of variables and secrets	Automatable after manual enable/custom trigger, depends on workspace variables and external secrets
Self-hosted Kubernetes runners configuration	Convenient deployment using two Helm charts	Easiest deployment using single Helm chart	The most complex deployment using Kubernetes resources, biggest customization
Free quota exceeding behaviour	Pay-as-you-go billing or all workflows are put in waiting	Some jobs are blocked until new minutes are bought	All pipelines are blocked until new minute packs are bought
Price for additional 1000 compute minutes	6\$	10\$	10\$

4.2 Performance evaluation of CI/CD pipelines

The last subchapter of this thesis describes and analyses numerical data collected across CI/CD pipelines run in GitHub, GitLab and Bitbucket. The method used for calculating all statistical parameters for this part is described in Section 3.4. This part starts from analysing overall pipeline duration times with respect to criteria like, runner hosting infrastructure and code base size and answers thesis question: *how pipeline execution time and start latency depend on code base size and runner environment*.



(a) Average pipeline duration by provider and repository size



(b) Average pipeline duration by provider and runner environment

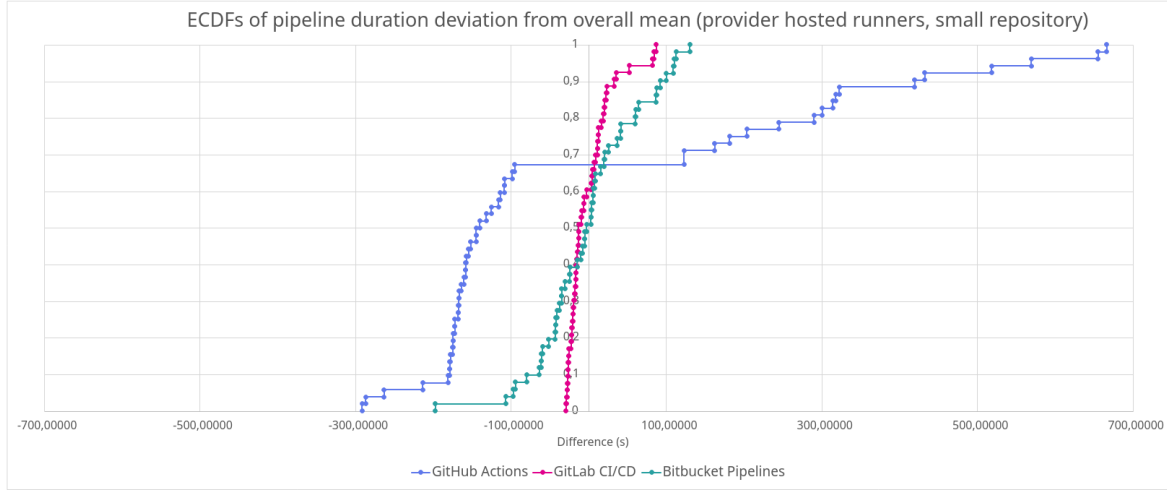
Figure 16. Average pipeline durations by provider

Figure 16 presents average pipeline durations by provider and repository size in Subfigure 16a, and by runner environment in Subfigure 16b. These bar charts average the pipeline duration metric, which is measured from the moment when the first command in the pipeline is available to run (pipeline start time) to the moment when the last command in the pipeline is available to run, excluding the sending of the data (pipeline stop time). The pipeline duration metrics in general do not focus on any specific time interval, but rather aggregate all measurements.

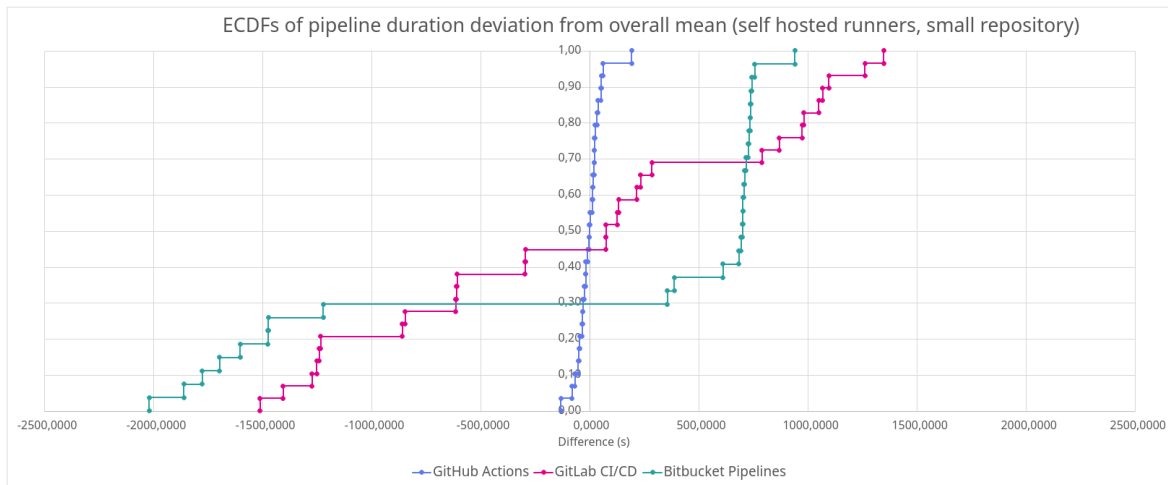
Since running pipelines either with provider-hosted runners or self-hosted runners incurs costs, the data population is limited to a few dozen per scenario. Each data point in this population is a pipeline duration in seconds. For GitHub, there are 51 data points for the small repository and provider-hosted runners, 29 data points for the small repository and self-hosted runners, 29 data points for the large repository and provider-hosted runners, and 31 data points for the large repository and self-hosted runners. For GitLab, there are 53 data points for the small repository and provider-hosted runners, 30 data points for the small repository and self-hosted runners, 29 data points for the large repository and provider-hosted runners, and 32 data points for the large repository and self-hosted runners. For Bitbucket, there are 51 data points for the small repository and provider-hosted runners, 27 data points for the small repository and self-hosted runners, 29 data points for the large repository and provider-hosted runners, and 30 data points for the large repository and self-hosted runners. On the x-axis there are four categories: the unweighted mean across providers and each provider. The y-axis presents the arithmetic average (mean) of pipeline duration time in seconds.

The bars show that GitHub Actions tend to perform best, with only large repository runs being longer than the general average. Next is GitLab CI/CD, with small repository and self-hosted runs being longer than the general average. Last is Bitbucket Pipelines, with only large repository runs being below the general average. While average pipeline durations do not differ dramatically, with respect to the unweighted mean across providers, between providers, there are two other aspects which are very noticeable on the charts. The first - less obvious - is that GitHub's average pipeline duration for small repositories is almost twice as low as it is for GitLab and Bitbucket and this factor is explained further in this subchapter. The second - more obvious - is that while it is expected for a large repository to have longer runs than a small repository - after all, building the Rust compiler from scratch is naturally a longer process than just a single-file Golang binary - it is not that common to say that self-hosted runs are very much slower than provider-hosted runs, with the difference in average being around 1000 seconds, or 16 minutes. The most important factor that may contribute to this phenomenon regards how self-hosted measurements are made in this thesis. For self-hosted workflows, a significant number of pipelines is run simultaneously for each provider, which creates a situation where tens of Kubernetes pods compete for cluster resources, which is always less powerful than provider-hosted infrastructure. The reason why, in this thesis, self-hosted runs always try to overload the Kubernetes cluster is that it is chosen as a way to create a significant queue of pipeline runs to test queueing-theory-related hypotheses. Therefore, the results do not support the conclusion

that self-hosted pipelines are, on average, slower than provider-hosted pipelines. More detailed further analysis allows for more far-reaching conclusions to be drawn regarding CI/CD pipelines.



(a) ECDFs of pipeline duration deviation from overall mean (provider-hosted runners, small repository)



(b) ECDFs of pipeline duration deviation from overall mean (self-hosted runners, small repository)

Figure 17. ECDFs of pipeline duration deviation from overall mean for small repository

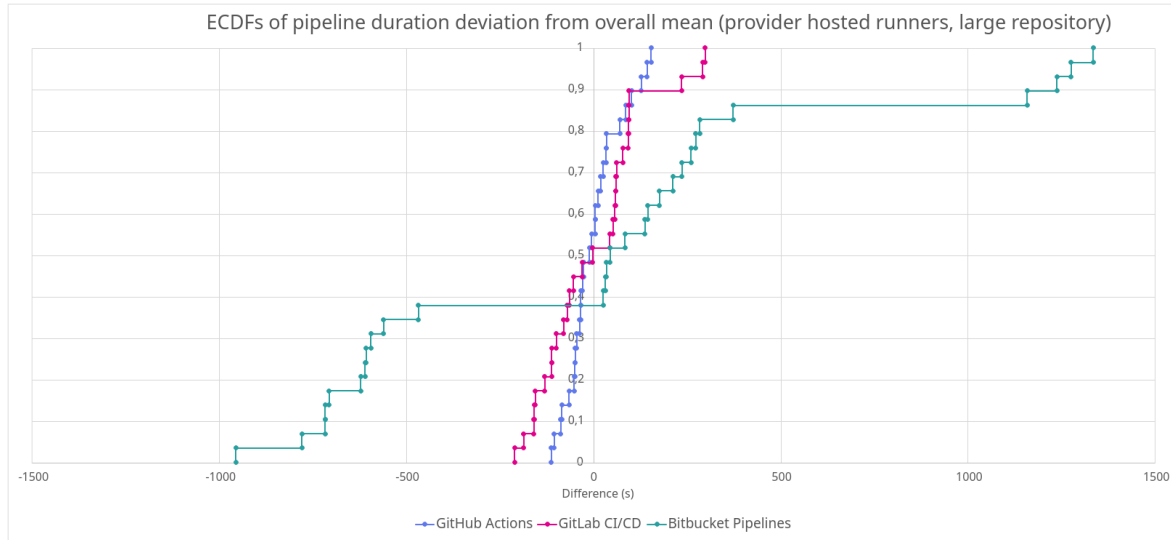
Figure 17 presents Empirical Cumulative Distribution Function (ECDF)s of pipeline duration deviation from overall mean for two of the four categories, which are provider-hosted runners with small repository, self-hosted runners with small repository, provider-hosted runners with large repository, self-hosted runners with large repository. For each population, the chart shows the ECDF of the centered pipeline durations $d_i = x_i - \bar{x}$, where x_i is the pipeline duration and $\bar{x}$ is the arithmetic mean of that population. The d_i values are sorted in ascending order, and the ECDF at threshold t gives the fraction of runs with $d_i \leq t$. The pipeline duration dataset presented on these Figures is

the same as it is for previous bar charts and therefore exact population numbers for each category per provider are described there. The x-axis presents the deviation of pipeline duration from the arithmetic mean in seconds. Negative values indicate pipeline runs which are faster than the mean and positive values indicate runs that are slower than the mean. The y-axis presents values of the distribution function in fractional percents. For example for GitHub small repository and provider-hosted runners 50% or 0.5 of all pipeline runs were faster than average for at least 140 seconds.

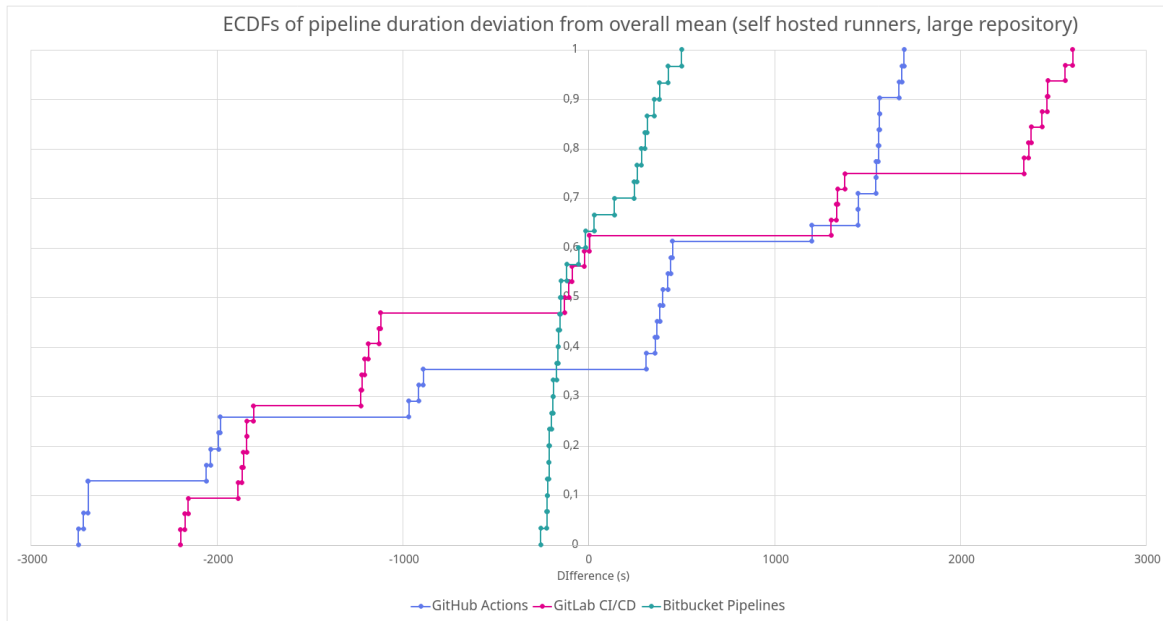
As far as the first category is concerned, it is presented in the Subfigure 17a. For small repositories and provider-hosted runners pipelines are run a few simultaneously – about 10. Under these conditions GitLab and Bitbucket show stable and centered distributions, while GitHub shows two clear distributions – one for about 67% of runs and the other for about 33% of runs. This clearly shows instability of GitHub pipeline infrastructure for this population of runs, with most pipelines finishing at least 100 seconds faster than average – first, leftmost distribution – and a significant group of pipelines finishing in a range from 100 to almost 700 seconds above average – second, rightmost distribution. This distribution split is arguably caused by the fact, that GitHub might assign less powerful pipeline run environments for Free plans and some pipelines might be queued longer resulting in this second, worse performing group. GitLab and Bitbucket, on the other hand, present visible single distributions for their pipeline durations. Almost all pipeline runs for both providers finish within the ± 100 seconds to their respective averages with GitLab performing in the most stable manner of all three providers.

Next chart, presented in the Subfigure 17b, regarding self-hosted pipelines and small repository looks noticeably more unstable, which is expected due to the nature of running pipelines in an almost overloaded Kubernetes cluster. In this setup GitHub turned out to be the most stable platform with almost all pipeline runs being in the range of ± 132 seconds of the average. GitLab CI/CD while spanning its pipeline duration in the range of about ± 1500 seconds of the average, and therefore offering the least stability, shows behaviour consistent with a single distribution. Last, but not least, Bitbucket pipelines chart shows two distribution groups, the first for about 30% of the fastest runs and a second group, that contains vast majority of pipelines running for longer than 350 seconds. This distribution characteristic for each provider is most likely caused by the work characteristics of each runner set architecture on an intentionally saturated Kubernetes cluster. GitHub's Actions Runner Controller benefits from its event-driven dispatch. A listener can react promptly to new jobs and request runner creation without waiting for a coarse polling cycle, so the dominant latency comes from cluster-level delays rather than slow control-plane detection. GitLab's Kubernetes executor, on the other hand, provisions a separate pod per job which, when many pipelines are started at once, creates a lot of short-lived tasks competing for the same limited compute and startup capacity, leading to larger variability and a wider spread of observed times. Finally, Bitbucket's runners-autoscaler in this experiment is poll-based with 10-minute polling, so when the single warm runner is occupied, additional capacity may be added only after the next polling interval, and then still competes for

scarce cluster resources. This combination naturally produces heavier tails and occasional extreme delays, which can raise the arithmetic mean substantially even if the central part of the ECDF appears only moderately wider.



(a) ECDFs of pipeline duration deviation from overall mean (provider-hosted runners, large repository)

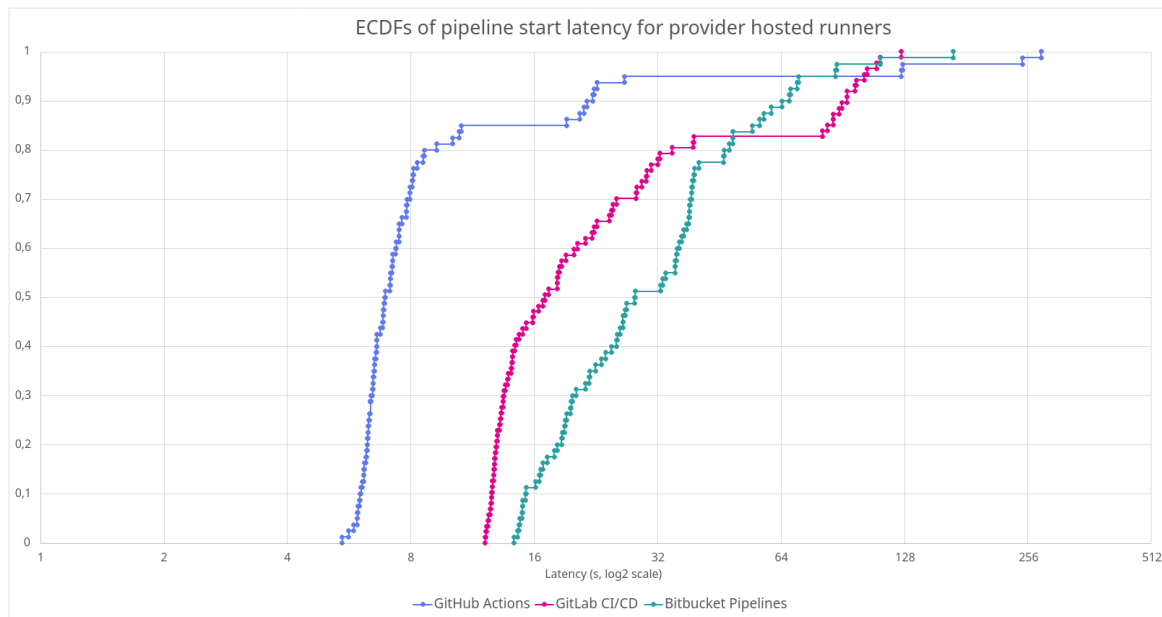


(b) ECDFs of pipeline duration deviation from overall mean (self-hosted runners, large repository)

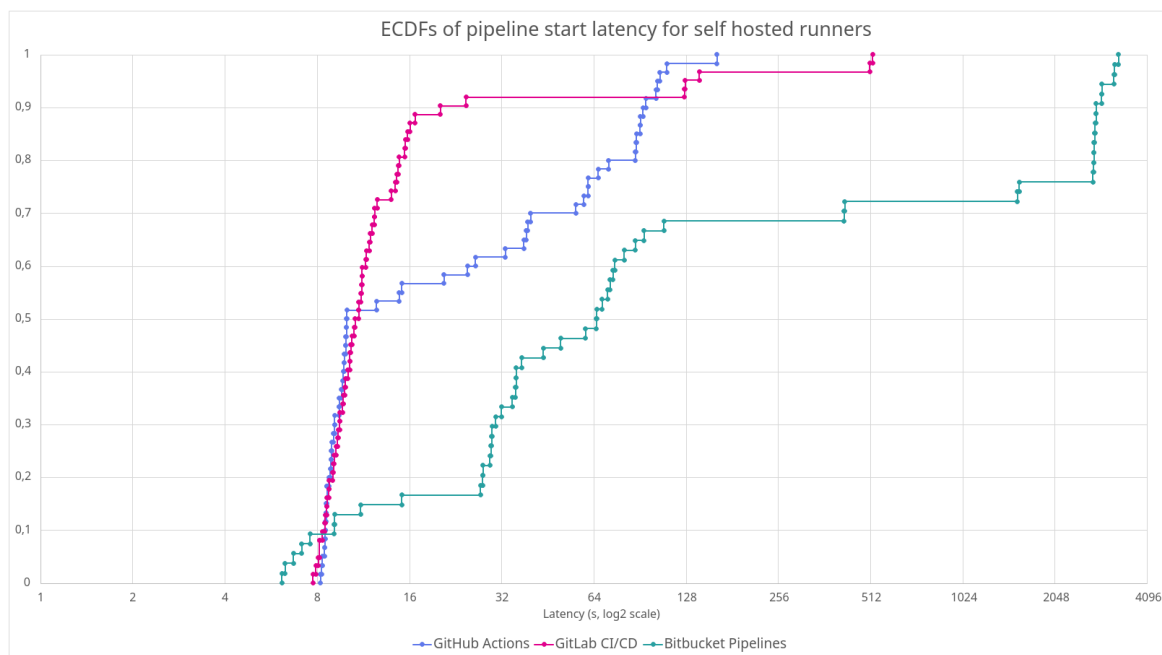
Figure 18. ECDFs of pipeline duration deviation from overall mean for large repository

Figure 18 presents ECDFs of pipeline duration deviation from overall mean for the remaining two of the four categories. The first one, presented in the Subfigure 18a, shows distributions of pipeline

duration deviation from overall mean for large repository and provider-hosted runners. Pipeline in large repository that is considered in this analysis builds Rust compiler from source code, which itself is a time-consuming task and therefore tests very well pipelines in terms of long runs. In this category it is clear, that both GitHub and GitLab offer the most stable run durations for long pipelines with small deviation from average of about ± 150 seconds for GitHub and deviation range of approximately $(-210, 300)$ seconds for GitLab. Bitbucket performed the worst in the category with pipeline duration deviation from average being in the range of approximately $(-950, 1335)$ seconds and visible three regions of performance. There might be multiple reasons behind this result, such as temporary network connectivity. However, the most convincing explanation regards runner underlying machine heterogeneity for Bitbucket Free plan which, if it were true, starts to significantly impact pipeline performance stability on longer runs. The last Subfigure 18b, discussed in this paragraph, regards pipeline duration deviation from overall mean for large repository and self-hosted runners. On the contrary, Bitbucket performs best in this category with pipeline duration deviation presenting most stable distribution and the lowest variance of all – in the range of approximately $(-250, 500)$ seconds. Both GitHub and GitLab present extremely unstable distributions with 6 distribution groups for GitLab and 5 distribution groups for GitHub, creating noticeable “steps” in the charts. At this level of variance, in comparison with Bitbucket, it is inconclusive to find a single explanation for this specific distribution for GitHub and GitLab pipeline runs. As the machines in the Kubernetes cluster for self-hosted workloads are homogeneous – meaning all compute instances have the same specifications – and it is not possible to conclude, that GitHub and GitLab pipeline runs have characteristic “steps” in the ECDFs for that reason. On the contrary, this factor might actually prove, that for homogeneous compute instances Bitbucket does not show any noticeable dispersions in the distribution function.



(a) ECDFs of pipeline latency for provider-hosted runners



(b) ECDFs of pipeline latency for self-hosted runners

Figure 19. ECDFs of pipeline latency

Figure 19 shows ECDFs of pipeline latency for two execution categories. The first category, presented in the Subfigure 19a, covers pipelines running on provider-hosted infrastructure, where the CI/CD platform provisions and manages the runners. The second category, presented in the Subfigure 19b,

covers pipelines executed on self-hosted runners, where the execution environment is controlled by the user. Because latency in this analysis is primarily driven by runner availability, scheduling, and the underlying compute environment rather than the project's codebase, comparing results across different repository sizes is not particularly informative for this metric. Accordingly, the latency ECDFs are reported without separating the data by repository size. In this research, pipeline latency is primarily shaped by several factors that differ in practical significance across GitHub Actions, GitLab CI/CD, and Bitbucket Pipelines. The network connectivity between the workstation that initiates pipeline run and the provider, as well as the network connectivity between the provider and the data centre, are observed to be largely insignificant contributors, typically remaining within sub-seconds latency and therefore not materially affecting the overall latency distribution. In contrast, the scheduling of runner resources introduces a slightly more significant component - commonly up-to-seconds latency - while the most significant driver is the time a pipeline run spends waiting in queue, which dominates end-to-end latency and explains the largest deviations between providers and execution environments.

The dataset for this comparison is computed by subtracting each pipeline start time from pipeline call time, which are metrics explained earlier, and then grouping them together by runner environment. These metrics come from the same pipeline runs as pipeline duration metric and therefore the number of data points maps directly to the amount of data described in the pipeline duration. Namely for provider-hosted environments there are 80 data points for GitHub, 82 data points for GitLab and 80 data points for Bitbucket. For self-hosted environments there are 60 data points for GitHub, 62 data points for GitLab and 57 for Bitbucket. The x-axis presents pipeline start latency in seconds, scaled in binary logarithmic scale. This choice allows the plots to capture outliers as well, which can indicate the worst-case delays observed for each provider, while still preserving readability in the typical latency range. The y-axis presents values of the distribution function in fractional percents.

The first subfigure 19a shows pipeline latency ECDF for provider-hosted pipeline runs. In this category GitHub's latency is very noticeably superior over other providers in the latency range below 64 seconds with about 95% of pipeline runs having lower latency than 32 seconds. For GitHub latency values over 64 seconds can be described as outliers, which sometimes occur in repositories where large amount of pipelines runs in a continuous way. The largest registered latency value for GitHub in this study is approximately 276 seconds (above 4 minutes). The lowest latency score for GitHub is about 5 seconds. GitLab in this category has the second lowest latency with approximately 82% of pipeline runs being below 39 seconds. Above 39 seconds latency Bitbucket has less latency than GitLab with 95% of runs for Bitbucket having lower latency than 70 seconds, while GitLab for 95% of pipeline runs has lower latency than about 103 seconds. The largest recorded latency for GitLab in this thesis is approximately 126 seconds, while the lowest recorded latency is about 12 seconds. Bitbucket has the biggest latency in the range up to about 48 seconds. The largest recorded latency for Bitbucket in this thesis is about 169 seconds, while the lowest recorded latency is 14 seconds.

The second subfigure 19b shows pipeline latency ECDF for self-hosted pipeline runs. One of the most visible differences between the provider-hosted chart and the self-hosted chart is the fact that x-axis for self-hosted pipelines ends at 4096 seconds (above an hour), while for provider-hosted it ends at 512 seconds (8 times less). This proves, that the most significant factor impacting pipeline start latency is the amount of time a pipeline needs to wait in a queue, since as mentioned earlier, Kubernetes cluster in this study is purposefully on the verge of overloading, so as to better test queueing parameters. In this category, GitLab CI/CD is the solution that offers the best performance, when it comes to pipeline start latency, with about 91% of pipeline runs starting under 24 seconds. The largest latency recorded in this thesis for GitLab is about 521 seconds, while the lowest latency recorded is about 7 seconds. GitHub is the second fastest starting pipelines solution in this study. Together with GitLab, most pipeline runs start in just under 10 seconds. For GitHub about 91% of pipeline runs start under 94 seconds. The largest recorded pipeline start latency for GitHub in this thesis is approximately 162 seconds, while the fastest pipeline start is about 8 seconds. Bitbucket Pipelines exhibits the largest latencies in this comparison. For Bitbucket, pipeline start latency spans from roughly 6 seconds up to 3306 seconds (about 55 minutes), with the most extreme values forming a pronounced tail that begins around 2731 seconds.

As mentioned earlier, the most important factor when it comes to pipeline start latency is arguably the time pipeline call waits in the queue on the provider's side. Therefore, it is reasonable to frame both subfigures as showing which provider empties their queue faster based on the runner environment. For provider-hosted runners, each provider's chart resembles a standard unimodal distribution with only a few intentionally omitted outliers. A plausible explanation is that provider-hosted capacity is elastic and not artificially constrained by user-managed infrastructure, which enables more uniform queue draining - within the limits imposed by free-tier allocations. In contrast, the self-hosted runner charts span a much wider time range and display noticeably more irregular distributions. This is consistent with the Kubernetes cluster operating near saturation. As the cluster approaches overload, additional runners cannot be provisioned quickly enough, and the queue consequently empties more slowly.

Table 3. Extended pipeline run metrics for a defined close time interval for GitHub small repository and provider-hosted runner environment

Arrival time t_i	Pipeline start time t_{s_i}	Inter-arrival time A_i [s]	Service time S_i [s]
20:01:15.529452000	20:01:23.144178905	–	999,87
20:01:20.186725000	20:01:26.424173351	4,66	1012,72
20:01:24.174084000	20:01:30.228126896	3,99	1098,34
20:01:28.162471000	20:01:34.338088896	3,99	1149,69
20:01:32.043541000	20:01:38.367573624	3,88	1235,40
20:01:36.108837000	20:01:42.499972751	4,07	1246,50
20:01:40.537481000	20:01:47.926070328	4,43	897,86

The last part of the performance evaluation study answers the last research question of this thesis, namely how observed queueing metrics relate to M/G/1, G/G/1 and G/G/c models for different arrival patterns. In particular, the research focuses on more specific problems, such as how well CI/CD pipelines for each provider can be described using M/G/1 and G/G/1 models, by abstracting provider infrastructure as a single queue emptying server, and also what arrival patterns must occur in pipeline invocations so that the queueing theory could be applied. At first, arrival patterns are analysed, where pipeline calls occur very often, that is within few seconds. Such a pattern is presented in the Table 3. This table presents pipeline run metrics, that relate to a series of GitHub small repository and provider-hosted pipeline invocations, which happened within approximately 4 seconds each. This naturally created a queue of runs, which executed with some service time S . The first question is whether this pipeline run series can be described by any queueing model previously mentioned, starting from the M/G/1 model. In particular this study focuses on estimating expected wait time in queue $\mathbb{E}[W_q]$ from pipeline call arrival times t_i and respective service times S_i . All formulas in this part are covered in the Section 2.4. From the data, presented in Table 3, expected (mean) service time $\mathbb{E}[S]$ is calculated and is approximately 1123.75 and expected (mean) arrival time $\mathbb{E}[A]$ is 4.17. Based on this data expected wait times in this queue are calculated with respect to all three models.

Table 4. Calculated expected wait in queue times from the Table 3 - see Section 2.4 for exact formulas

Queue model	Expected wait in queue time [s]
$\mathbb{E}[W_q]_{M/G/1}$	-571.43171
$\mathbb{E}[W_q]_{G/G/1}$	-10.48279

For the data from the Table 3, expected wait in queue times are presented in the Table 4. For M/G/1 model this value is -571.43171 seconds, while for model G/G/1, which considers more general arrival patterns expected wait in queue time is -10.48279 seconds. Both values prove, this dataset does not present stable M/G/1 or G/G/1 queue models, because negative expected wait in queue time values are physically non-existent. Queues in queueing theory model stable systems, meaning all formulas used there work only for queue growth matching serving capabilities - capacity must exceed demand. Negative $\mathbb{E}[W_q]$ in queueing theory most often means, queue is experiencing such a growth in service requests, that exceeds servers capacity. In this example expected wait in queue times for both models are negative because of the $\frac{\rho}{1-\rho}$ factor - see Section 2.4 for exact formulas, which becomes negative for $\rho > 1$. ρ represents a single-server utilisation in queueing theory and is equivalent to a multiplication of mean arrival rate λ and expected service time $\mathbb{E}[S]$. Because $\lambda = \frac{1}{\mathbb{E}[A]}$, following ρ formula is valid.

$$\rho = \lambda \cdot \mathbb{E}[S] = \frac{\mathbb{E}[S]}{\mathbb{E}[A]} \quad (7)$$

Where:

1. $\lambda = \frac{1}{\mathbb{E}[A]}$ - is customer arrival rate
2. $\mathbb{E}[S]$ - is expected (mean) service time
3. $\mathbb{E}[A]$ - is expected (mean) inter-arrival time

A queue is stable for $\rho \in (0, 1)$, therefore based on the Eq. 7 following relation $\mathbb{E}[A] > \mathbb{E}[S]$ is true. For pipeline run invocation this means, that pipelines on average should be triggered with longer time intervals, than servers take time, on average, to process individual pipeline. For models considering multiple servers available to serve the queue, lambda is divided by c - number of active servers. For example, assuming the dataset from the Table 3 describes G/G/c queue model, it is possible to estimate number of required pipeline runners, that must be actively running pipelines from the queue, to be able to consider it a G/G/c queue model.

$$\rho_c = \frac{\lambda}{c} \cdot \mathbb{E}[S] = \frac{\mathbb{E}[S]}{c \cdot \mathbb{E}[A]} \quad (8)$$

$$c_{\min} = \left\lceil \frac{\mathbb{E}[S]}{\mathbb{E}[A]} \right\rceil, \rho_c = 1 \quad (9)$$

Where:

1. ρ_c - single server utilisation for multi-server queues
2. $\lambda = \frac{1}{\mathbb{E}[A]}$ - is customer arrival rate
3. c - is the number of servers available in the queue
4. $\mathbb{E}[S]$ - is expected (mean) service time
5. $\mathbb{E}[A]$ - is expected (mean) inter-arrival time

The minimum number of servers required to ensure queue stability, $c_{\min}$, is given by Eq. 9 and equals 297. This implies that, to interpret this dataset as a sample from a stable G/G/c queue, the system would require at least 297 concurrently available pipeline runners. Such a runner count is never reached at any point in this study. Consequently, for inter-arrival times A_i on the order of only a few

seconds, the classical steady-state queueing models M/G/1, G/G/1, and G/G/c are not applicable to the observed pipeline runs for any provider, regardless of repository size or runner environment. These models become meaningful only under arrival rates that satisfy the stability condition implied by the measured service times, for instance when pipeline calls are sufficiently infrequent so that $\rho < 1$. To test this additional pipeline run data is collected, which satisfies following requirements. The new data is collected by running for each provider pipelines from small repository and large repository 8 times. Data from small repository regards provider-hosted pipelines, while the data from large repository – self-hosted infrastructure. The frequency of pipeline calling is estimated from existing data knowing, that the longer the inter-arrival times, the more reliable the results, since this satisfies the most important requirement, that capacity must exceed demand. Therefore service times (pipeline duration times) S are analysed for each provider and for each category the longest service time recorded for all providers is estimated as the worst-case expected service time $\mathbb{E}_{\max}[S]$. For provider-hosted pipelines, the number of running servers assumed is 1, which should test if such a system can be modelled using M/G/1 or G/G/1 and even more significantly expands inter-arrival times. For self-hosted pipelines, the number of runners is set to be about 4 at a time for each provider. Following queue stability formula is derived from Eq. 8 and $\rho > 1$.

$$\mathbb{E}[A] > \frac{\mathbb{E}[S]}{c} \quad (10)$$

Where:

1. c - is the number of servers available in the queue
2. $\mathbb{E}[S]$ - is expected (mean) service time
3. $\mathbb{E}[A]$ - is expected (mean) inter-arrival time

Given $\mathbb{E}_{\max}[S] \approx 1247s$ for provider-hosted pipelines and $\mathbb{E}_{\max}[S] \approx 6075s$ for self-hosted pipelines, $\mathbb{E}[A]$ for provider-hosted pipelines should be at least 1247s (above 21 minutes), while for self-hosted pipelines it should be at least 1519s (above 26 minutes). Given such assumptions, expected wait times are calculated for each provider and each queueing model, and are gathered in Tables 5 and 6.

Table 5. Calculated expected wait in queue times for provider-hosted pipeline runs meeting criterion defined in Eq. 10 - see Section 2.4 for exact formulas

Queue model	GitHub Actions [s]	GitLab CI/CD [s]	Bitbucket Pipelines [s]
$\mathbb{E}[W_q]_{M/G/1}$	87.98679	116.49763	118.31632
$\mathbb{E}[W_q]_{G/G/1}$	1,63291	0,01881	1,25982
$\mathbb{E}[W_q]_{G/G/2}$	0.08876	0.00110	0.07348
$\mathbb{E}[W_q]_{G/G/3}$	0.00538	0.00007	0.00495

Table 5 presents calculated expected wait in queue times for pipeline runs meeting criterion defined in Eq. 10. All values for each provider are positive with $\mathbb{E}[W_q]_{M/G/1}$ wait times being significantly higher than for $\mathbb{E}[W_q]_{G/G/1}$. This shows, that all providers provided pipeline runs with stable queueing mechanism, and that all models showed reasonable expected wait times in $\mathbb{E}[W_q]_{G/G/c}$ queues – for as much as the very limited nature of this auxiliary experiment allows - which might suggest such arrival characteristic is better modelled by $\mathbb{E}[W_q]_{G/G/c}$ queues rather than by $\mathbb{E}[W_q]_{M/G/1}$ model. This also shows, that under specific conditions queueing models M/G/1, G/G/1 and G/G/c can be used to describe CI/CD pipeline queueing characteristics, provided that $\mathbb{E}[A]$ is significantly greater than $\mathbb{E}[S]$. The M/G/1 model shows expected wait in queue times that are significantly longer than average pipeline latency for the same dataset. M/G/1 assumes Poisson arrivals and $C_A^2 = 1$. Even with low average utilization, Poisson arrivals create occasional arrival bursts, which increases theoretical waiting. The reason why these results differ substantially from the measured pipeline latency is that, in this study, pipeline latency is defined as the time from the API pipeline call to the first effective command executed inside the pipeline. Most of this latency is not caused by queueing itself, but by the delay between the runner picking up the job and preparing the execution environment required for the pipeline to run successfully. In this case queueing models arguably assume the service time starts with customer leaving the queue, which is not the case for CI/CD pipelines.

Table 6. Calculated expected wait in queue times for self-hosted pipeline runs meeting criterion defined in Eq. 10 - see Section 2.4 for exact formulas

Queue model	GitHub Actions [s]	GitLab CI/CD [s]	Bitbucket Pipelines [s]
$\mathbb{E}[W_q]_{M/G/1}$	-5034.13069332	-4990.98380505	-3207.22166077
$\mathbb{E}[W_q]_{G/G/1}$	-822,5425691	-812,5747951	-8,917384874
$\mathbb{E}[W_q]_{G/G/2}$	-821.9686944	-815.2812038	15.82919367
$\mathbb{E}[W_q]_{G/G/5}$	347.0036917	283.4903479	0.048604677

Subsequently, queueing model for self-hosted runners proved to be even more complex, than provider-hosted version, which is visible in the Table 6. For models M/G/1 and G/G/1 all $\mathbb{E}[W_q]$ values are negative, which is reasonable since this auxiliary experiment is designed in a way to make sure multiple runners are engaged – preferably 4 at a time. Apart from that, results from previous part of this subsection show, that self-hosted runners are in general less stable, which explains negative values. However, expected wait times for G/G/c model have different characteristics. For number of runners in an expected range of up to 4, $\mathbb{E}[W_q]$ is mostly negative with the only exception of Bitbucket, which has negative expected wait in queue time only for $c = 1$. For $c > 4$ all providers have positive $\mathbb{E}[W_q]$ values with some of them being in reasonable ranges around measured pipeline latency. This shows, that while the actual results are generated from pipelines queued in an unstable (from the perspective of these queueing theory models) queue, this stability can be easily reached when increasing the number of servers – or pipeline runners.

Overall, queueing theory while useful for analysing pipelines run in time intervals larger, than the actual pipeline duration, is impractical for analysing queueing characteristics of CI/CD pipelines run in large bursts or only busier environments. Under bursty invocation patterns with inter-arrival times of only a few seconds, the estimated utilisation exceeds one, causing M/G/1 and G/G/1 (and, unless an unrealistically large c is assumed, also G/G/c) to produce invalid negative waiting-time estimates. This leads to the conclusion, that queue stability is necessary for the analysis to be based on any of the aforementioned models. In the auxiliary stable experiments with extended inter-arrival times, all providers produced reasonable, non-negative expected waiting times, indicating that - under such conditions - queueing theory can be used to characterise pipeline queueing behaviour. However, this auxiliary experiment is based on a very small statistical population, which means the estimated parameters and model outputs should be treated as indicative rather than statistically conclusive, and the results are best interpreted as a qualitative validation of applicability under stable arrival conditions rather than a precise quantitative characterisation.

Chapter 5

Summary

The Summary chapter goes through both practical and observations part of the thesis and in particular outlines all important findings, and relates them to the objectives and questions stated in the introduction.

The text starts by briefly answering the questions from the introduction. Then, it discusses what is configured and created in the implementation section, with emphasis on how it is designed, so as to provide comparable testing environments for each of the providers, and continues with describing how actual pipeline definitions are created for each provider with a general overview of the solution, that collects all the pipeline results and performs simple transformations on hundreds of entries. Next, the chapter briefly summarises all the important findings and conclusions, both subjectively, from the experience of running multiple and complex pipelines, and objectively based on the actual metrics calculated from the actual dataset, with emphasis on pipeline duration, pipeline latency and expected wait in queue time - from the mathematical queueing theory.

Finally, the chapter discusses limitations of the study. It explains the assumptions made when constructing the experimental environment, the selected test workloads, and the characteristics of the underlying infrastructure, and indicates how these design decisions may affect the scope of the conclusions. The closing section introduces possible extensions of the work, including broader classes of applications, additional CI/CD features, or other hosting services, and suggests how the methodology developed in the thesis can be reused or adapted.

The study questions in this thesis regard three fundamental questions, namely how the platforms differ in configuration model and ease of use for typical project workflow, how pipeline execution time and start latency depend on code base size and runner environment and lastly, how observed queueing metrics relate to $M/G/1$, $G/G/1$ and $G/G/c$ models for different arrival patterns.

The purpose of this thesis is not to point a single best CI/CD solution, but rather analyse potential advantages and disadvantages of each platform, and therefore the answer to all three questions is more nuanced. The first question answered in this thesis regards user-oriented issues with operating on CI/CD pipelines. In particular for each provider analyses pipeline log interface, reusability, automation capabilities, self-hosted runner environment configuration and architecture, free quota exceeding behaviour and a price estimate for additional 1000 compute minutes. The second question is answered by analysing datasets collected for CI/CD pipeline runs for each provider with respect to repository code base size and place of runner hosting. This subsection analyses pipeline runs in terms of how long pipeline works once it starts, and how long the user waits before it is possible to execute first command of the pipeline. The last part regards mathematical queueing theory and its application to CI/CD pipelines. While presented queueing theory models can be a useful for CI/CD, it is only true under conditions that resemble stable service systems. For bursty pipeline call patterns with short inter-arrival times, those steady-state models cease to describe the observed measurements in any meaningful way, unless the number of servers picking customers from queue increases to a high-enough level to be able to efficiently serve pipeline runs in the given expected inter-arrival time.

The implementation work begins with the construction of a comparable testing environment, and this is the central theme of the Subchapter 3.1. The core idea is that each provider's environment must be as close to each other as possible, with the same organisational structure, the same repositories, and the same ability to evolve the experiment without silently drifting into incomparable configurations. For this reason, the thesis introduces a dedicated management repository that aims to deploy experimental environment using IaC approach, so that organisations, projects, repositories, and permissions can be recreated and modified at any point. The IaC approach allows to significantly reduce costs related with running self-hosted pipeline infrastructure, by removing and creating tens of cloud resources only for a short period of pipeline result collection with a single CLI command. Each provider is configured to contain two comparable code base repositories and one repository that stores shared pipeline templates, so that the same conceptual pipeline can be executed across providers. In practice, GitHub and GitLab are provisioned using Terraform modules written in HCL, which allows to easily define resources. At the same time, the work also highlights that "*declarative*" does not always mean "*fully automated*", because for example GitLab SaaS imposes constraints that require a manual creation of a top-level group before it can be imported and referenced by Terraform. Bitbucket, on the other hand, is configured using a Python module that calls Bitbucket's directly, because the ecosystem of stable, maintained Terraform providers for Bitbucket is not comparable with GitHub or GitLab, and therefore an API-driven approach is the most reliable way to keep the

Bitbucket side of the environment idempotent and controllable. Apart from that, while Bitbucket has pipeline template repository created, the free version of Bitbucket itself does not support pipeline sharing or reusing and for that reason this repository is unused, which is an only meaningful deviation from the general testing environment architecture.

The next layer of implementation, described in Subchapters 3.2, 3.3, and 3.4, moves from configuring providers to describing actual pipelines that generate test results and related infrastructure that transports and processes raw data from each pipeline run. Subchapter 3.2 defines two execution models for each provider, namely provider-hosted runners and self-hosted runners. It then applies both models to a small repository, which runs a short CI/CD pipeline, mostly compiling simple Golang application, that resembles a typical DevOps approach to a pipeline. The large repository pipeline runs, on the other hand, focus purely on measuring long-running pipeline efficiency, by building a Rust programming language compiler from code. This produces four distinct experimental scenarios per provider, which separates the influence of codebase size from the influence of runner environment. A crucial practical detail is that the pipelines are instrumented to capture timestamps such as the pipeline call time, the effective pipeline start time, and the stop time, and then they export a standardised JSONL file containing pipeline run results, so that performance analysis of many hundreds of files can be automated. In Subchapter 3.3, self-hosted runners are deployed on a Kubernetes cluster. Kubernetes is used only to host and scale the runners and is not treated as the main subject of the thesis. This is only the tool to enable easy scalability of pipeline runners for each provider and is regarded as a “*blackbox*” that brings just enough computing powers for each provider’s pipelines. The cluster is created in AWS using EKS and Terraform, and it is configured in a way that makes scaling behaviour visible under pressure, with a node group composed of 2 Spot instances of type t3.large, each offering 2 vCPUs and 8 GiB of memory, which is sufficient to demonstrate contention effects when many runner pods compete for finite resources. The last Subchapter 3.4 of the thesis introduces the last “*moving part*” of the whole pipeline testing solution, by introducing a central result files storage in AWS S3, and a lightweight analytical layer using AWS Glue and Athena, so that experimental results can be easily queried and simple mathematical formulas are calculated this way.

The qualitative results are placed in the Subchapter 4.1. This part subjectively compares various aspects of operating on CI/CD pipelines across each provider based on actual functionalities, provider characteristics and overall experience of operating on the pipelines during actual test results collection. Each provider has its individual set of advantages and disadvantages. GitHub Actions for example offer a clear separation of logs by job and step, which makes it easier to locate the failing phase and related errors, but can be problematic with maintaining tens of complex pipeline YAML pipeline definitions and templates in a single repository. On the contrary, GitLab tends to concentrate information into a continuous job log stream that can be rich in metadata, but harder to read and debug, yet provides the most flexibility when it comes to pipeline reusability and arguably the

easiest self-hosted runners implementation. Last, but not least, Bitbucket while possessing the benefit of being integrated in an Atlassian tool suite, does not perform outstandingly well when it comes to aspects analysed in this part with pipeline reusability being present in only higher plans, pipeline automation capabilities being limited by first pipeline run in the repository, which must be run manually from the webpage and the most complex Kubernetes self-hosted runners implementation.

The quantitative results in Subchapter 4.2 then translate the same platforms into measurable behaviour, beginning with pipeline duration and pipeline start latency, and finishing with an explicit attempt to relate observed queueing metrics to selected queueing models from the queueing theory. It first compares overall averages across providers, by repository sizes, and runner environments. Average pipeline durations across providers do not differ dramatically when aggregated, however two conclusions are drawn from these bar charts. The first conclusion is that for small repositories, GitHub's average pipeline duration is almost twice as low as GitLab's and Bitbucket's, The second conclusion is that for self-hosted runs the average can appear around 1000 seconds (about 16 minutes) slower than for provider-hosted runs, which is explained by the assumptions made in the study regarding oversaturation of the Kubernetes cluster to supposedly test queueing characteristics. It is important to state, that most self-hosted pipeline runners implementations in the industry rarely ever experience such stress levels, which potentially could help drawing new conclusions. To move beyond averages, the study analyses ECDFs of pipeline duration deviation from the mean, which reveals stability properties for each provider. For small repositories on provider-hosted runners, GitLab and Bitbucket exhibit stable, centered distributions, while GitHub shows a split into two distinct performance groups. For large repositories on provider-hosted runners, GitHub and GitLab show relatively small deviation ranges, while Bitbucket shows much wider deviation, which is arguably associated with greater underlying machine heterogeneity for the Bitbucket Free pricing plan. Next, the analysis of pipeline start latency is described. For provider-hosted runners, about 95% of GitHub runs start below 32 seconds, with a lowest observed latency around 5 seconds and an extreme outlier around 276 seconds. GitLab shows around 82% of runs below 39 seconds and a maximum around 126 seconds. Lastly, Bitbucket shows higher latencies in the lower range, with a minimum around 14 seconds and a maximum around 169 seconds. For self-hosted runners, the latency scale stretches dramatically, with the x-axis reaching 4096 seconds, illustrating that the queue and scaling mechanisms of highly saturated Kubernetes cluster impact the most in this example. GitLab performs best in this category with about 91% of runs starting under 24 seconds and a maximum around 521 seconds. GitHub has pipeline latency in 91% of all observed runs under 94 seconds with a maximum around 162 seconds. Bitbucket in this category exhibits the heaviest tail with latencies up to about 3306 seconds. The last part of the subsection answers questions regarding comparing selected queueing theory models to actual observed pipeline results. For pipeline call arrival patterns with inter-arrival times of only a few seconds and pipeline durations counted in hundreds of seconds, server utilisation ρ exceeds 1, producing negative expected waiting times for models like M/G/1 and G/G/1, which is a clear sign that the stability assumptions of these steady-state models do not hold

for such workloads. This issue could be mitigated by dramatically increasing the number of available at a time runners and treating the whole implementation as a G/G/c system, but it would require hundreds of concurrent servers, which is a scale hardly ever reachable. To prove this conclusion, an auxiliary experiment is conducted with inter-arrival times estimated using queueing theory formulas and the models calculated reasonable expected waiting times, which supports the interpretation that queueing theory can describe CI/CD behaviour when the pipeline call rate stability conditions are met. There are ways the pipeline testing solution could be improved. First of all, the limited data population could be increased to make conclusions more grounded, however this comes at a significantly higher costs, especially for provider-hosted test results. While the thesis already collects dozens of runs per category, rare events in the far tails of distributions will always demand larger datasets for high statistical confidence about extreme percentiles. Another limitation is that the self-hosted environment is intentionally configured near saturation, which is beneficial for studying unstable queues. However, it also means the measured self-hosted latencies are not representative of a well-provisioned production cluster, but rather as a proof that self-hosted pipeline environments performance is related to the actual cluster performance. The queueing-theory analysis highlights another limitation. Bursty arrival patterns create a situation, where pipeline queue is no longer properly modelled by mentioned queueing theory models. A natural next step is to generate more test results that cover a richer variety of pipeline call arrival characteristics or perhaps to apply additional methods of analysing bursty patterns. Furthermore, CI/CD pipeline comparison could potentially be expanded by other solutions, either regarding comparing other Git hosting providers, which offer running such pipelines in provider and self-hosted environments, such as Azure DevOps platform, which is increasingly popular among teams, that work on a daily basis with the Microsoft Azure cloud services, or by extending the study to CI/CD platforms that integrate with external Git repositories and offer their own runner models, such as CircleCI, Travis CI, Buildkite, or Jenkins.

To sum up, the choice between providers and runner models significantly depends on each development team needs. For instance, if a team already works with several Atlassian tools and does not run heavy time or resource consuming pipelines, then such a team might greatly value Bitbucket as a Git hosting provider. If a team develops a large open-source project and needs stable pipeline execution platforms – either with provider or self-hosted pipeline environments – then GitHub might be the optimal choice. If a team however runs multiple and complex pipelines and works primarily with a DevOps methodology, then such a team might benefit the most from an almost unlimited pipeline reusability and a repository navigation divided into DevOps phases, which is offered by GitLab. Provider-hosted runners exchange control for ease and vice-versa for self-hosted environments. If an only requirement of a team is to occasionally run simple CI/CD pipelines, then picking specific provider does not entirely matter and all of the three providers meet modern requirements of teams regarding Git hosting platforms.

Bibliography

- [1] Rochkind, M. J., „The source code control system,” *IEEE Transactions on Software Engineering*, vol. SE-1, no. 4, pp. 364–370, 1975. DOI: 10.1109/TSE.1975.6312866.
- [2] Tichy, W. F., „Rcs—a system for version control,” *Software: Practice and Experience*, vol. 15, no. 7, pp. 637–654, 1985.
- [3] Chacon, S. and Straub, B., *Pro Git*, 2nd ed. Apress, 2014, ISBN: 9781484200773. Accessed: Dec. 13, 2025. [Online]. Available: <https://git-scm.com/book/en/v2>.
- [4] Gailly, J.-l. and Adler, M., „Zlib 1.3.1 manual,” zlib, Accessed: Dec. 13, 2025. [Online]. Available: <https://www.zlib.net/manual.html>.
- [5] „Git - git documentation.” Manual page (latest version listed on the page: 2.52.0), Git SCM / The Git Project, Accessed: Dec. 14, 2025. [Online]. Available: <https://git-scm.com/docs/git>.
- [6] „Gitflow workflow,” Atlassian, Accessed: Dec. 15, 2025. [Online]. Available: <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>.
- [7] „Git - git merge strategies.” Manual page (latest version listed on the page: 2.52.0), Git SCM / The Git Project, Accessed: Dec. 17, 2025. [Online]. Available: <https://git-scm.com/docs/merge-strategies>.
- [8] „Stack overflow developer survey 2022 - version control systems,” Stack Overflow, Accessed: Dec. 18, 2025. [Online]. Available: <https://survey.stackoverflow.co/2022/#version-control-version-control-system>.
- [9] „Continuous integration - definition.” Last modified September 20, 2023, CD Foundation, Accessed: Dec. 20, 2025. [Online]. Available: <https://bestpractices.cd.foundation/learn/ci/>.
- [10] „Continuous delivery - definition.” Last modified September 20, 2023, CD Foundation, Accessed: Dec. 20, 2025. [Online]. Available: <https://bestpractices.cd.foundation/learn/cd/>.
- [11] Marusiak, W. „How to start devops: A step-by-step guide for teams interested in devops.” English translation of the Polish-language Atlassian DevOps guide, Atlassian, Accessed: Dec. 20, 2025. [Online]. Available: <https://www.atlassian.com/pl/devops/what-is-devops/how-to-start-devops>.
- [12] „What is devops?” Accessed: 2025-12-20, Atlassian, Accessed: Dec. 20, 2025. [Online]. Available: <https://www.atlassian.com/devops>.

- [13] „What is a devops pipeline? a complete guide,” GitHub, Accessed: Dec. 21, 2025. [Online]. Available: <https://github.com/resources/articles/pipeline>.
- [14] „What is jenkins?” Jenkins Documentation, Jenkins, Accessed: Dec. 21, 2025. [Online]. Available: <https://www.jenkins.io/doc/#what-is-jenkins>.
- [15] „Stack overflow developer survey 2022 - version control platforms,” Stack Overflow, Accessed: Dec. 21, 2025. [Online]. Available: <https://survey.stackoverflow.co/2022/#section-version-control-version-control-platforms>.
- [16] „About github and git.” GitHub documentation page accessed on 2025-12-21, GitHub Docs, Accessed: Dec. 21, 2025. [Online]. Available: <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>.
- [17] „Github’s plans.” GitHub documentation page accessed on 2025-12-21, GitHub Docs, Accessed: Dec. 21, 2025. [Online]. Available: <https://docs.github.com/en/get-started/learning-about-github/githubs-plans>.
- [18] „Yaml: Yaml ain’t markup language.” Official YAML website, YAML Language Development Team, Accessed: Dec. 21, 2025. [Online]. Available: <https://yaml.org/>.
- [19] „Github actions documentation.” GitHub Actions documentation page accessed on 2025-12-21, GitHub Docs, Accessed: Dec. 21, 2025. [Online]. Available: <https://docs.github.com/en/actions>.
- [20] „Gitlab pricing.” Accessed: 2025-12-22, GitLab, Accessed: Dec. 22, 2025. [Online]. Available: <https://about.gitlab.com/pricing/>.
- [21] „What is a container?” Docker resource page accessed on 2025-12-22, Docker, Accessed: Dec. 22, 2025. [Online]. Available: <https://www.docker.com/resources/what-container/>.
- [22] Silvers, J. „Atlassian acquires bitbucket.org, distributed version control system hosting.” Work Life by Atlassian blog post, Atlassian, Accessed: Dec. 27, 2025. [Online]. Available: https://www.atlassian.com/blog/archives/atlassian_acquires_bitbucket_org_distributed_version_control_system_hosting%24.
- [23] „Bitbucket pricing.” Accessed: 2025-12-27, Atlassian, Accessed: Dec. 27, 2025. [Online]. Available: <https://www.atlassian.com/software/bitbucket/pricing>.
- [24] „What is snyk?” Snyk documentation page accessed on 2025-12-29, Snyk, Accessed: Dec. 29, 2025. [Online]. Available: <https://docs.snyk.io/discover-snyk/whats-snyk>.
- [25] Shortle, J. F., Thompson, J. M., Gross, D., and Harris, C. M., *Fundamentals of queueing theory*. John Wiley & Sons, 2018.
- [26] Kingman, J. F., „The single server queue in heavy traffic,” in *Mathematical Proceedings of the Cambridge Philosophical Society*, Cambridge University Press, vol. 57, 1960, pp. 902–904.
- [27] HashiCorp. „Terraform documentation,” Accessed: Jan. 6, 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>.

- [28] Terraform Registry. „Github provider documentation (integrations/github),” Accessed: Jan. 6, 2026. [Online]. Available: <https://registry.terraform.io/providers/integrations/github/latest/docs>.
- [29] Terraform Registry. „Gitlab provider documentation (gitlabhq/gitlab),” Accessed: Jan. 6, 2026. [Online]. Available: <https://registry.terraform.io/providers/gitlabhq/gitlab/latest/docs>.
- [30] Terraform Registry. „Gitlab_group resource — gitlab provider documentation (gitlabhq/gitlab),” Accessed: Jan. 7, 2026. [Online]. Available: <https://registry.terraform.io/providers/gitlabhq/gitlab/latest/docs/resources/group>.
- [31] Python Software Foundation. „Venv — creation of virtual environments,” Accessed: Jan. 7, 2026. [Online]. Available: <https://docs.python.org/3/library/venv.html>.
- [32] Hardt, D. „The oauth 2.0 authorization framework,” Accessed: Jan. 7, 2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>.
- [33] Atlassian Support. „Share pipelines configurations,” Accessed: Jan. 8, 2026. [Online]. Available: <https://support.atlassian.com/bitbucket-cloud/docs/share-pipelines-configurations/>.
- [34] Kubernetes Authors. „Kubernetes documentation,” Accessed: Jan. 8, 2026. [Online]. Available: <https://kubernetes.io/docs/home/>.
- [35] Terraform Registry, Anton Babenko. „Terraform-aws-modules/eks/aws — terraform module registry,” Accessed: Jan. 8, 2026. [Online]. Available: <https://registry.terraform.io/modules/terraform-aws-modules/eks/aws/latest>.
- [36] Terraform Registry, Anton Babenko. „Terraform-aws-modules/vpc/aws — terraform module registry,” Accessed: Jan. 8, 2026. [Online]. Available: <https://registry.terraform.io/modules/terraform-aws-modules/vpc/aws/latest>.
- [37] GitHub. „Deploy runner scale sets with actions runner controller,” Accessed: Jan. 8, 2026. [Online]. Available: <https://docs.github.com/en/actions/tutorials/use-actions-runner-controller/deploy-runner-scale-sets>.
- [38] GitLab. „Install gitlab runner on kubernetes (helm chart),” Accessed: Jan. 8, 2026. [Online]. Available: <https://docs.gitlab.com/runner/install/kubernetes/>.
- [39] Atlassian. „Autoscaler for runners on kubernetes,” Accessed: Jan. 8, 2026. [Online]. Available: <https://support.atlassian.com/bitbucket-cloud/docs/autoscaler-for-runners-on-kubernetes/>.

List of Abbreviations and Symbols

G/G/1 single-server queueing model with a general (arbitrary) arrival process, and general service-time distribution 37

G/G/c multiple-server queueing model with a general (arbitrary) arrival process, and general service-time distribution 37

M/G/1 single-server queueing model with Markovian (Poisson) arrivals, and general service-time distribution 37

API Application Programming Interface 34

AWS Amazon Web Services 54

CD Continuous Delivery 21

CI Continuous Integration 20

CI/CD Continuous Integration and Continuous Delivery 21

CLI Command Line Interface 17

CT Continuous Testing 21

CVCS Centralised Version Control System 14

DAST Dynamic Application Security Testing 21

DR Disaster Recovery 23

ECDF Empirical Cumulative Distribution Function 68

EKS Elastic Kubernetes Service 54

FIFO First In First Out 37

HCL HashiCorp Configuration Language 44

IaC Infrastructure as Code 23

IDE Integrated Development Environment 17

PAT Personal Access Token 46

PK *Pollaczek-Khintchine* 38

RCS Revision Control System 13

S3 Simple Storage Service 57

SaaS Software as a Service 24

SCCS Source Code Control System 12

VCS Version Control System 12

List of Figures

1	Local version control system scheme	13
2	Centralised version control system scheme	14
3	Distributed version control system scheme	15
4	Typical simple versioning scheme with Git	17
5	Simplified diagram presenting most common Git branching scenarios	18
6	Graphical depiction of the perpetual DevOps software lifecycle (the infinity loop) [11]	22
7	2022 "Stack Overflow" Developer Survey – version control hosting service popularity [15]	24
8	General queue model [25]	36
9	General testing environment architecture	42
10	Management repository file tree	43
11	General simple pipeline workflow	50
12	General Rust pipeline workflow	50
13	Test result data pipeline simple diagram	57
14	GitHub example pipeline logs	60
15	Example pipeline logs	61
16	Average pipeline durations by provider	66
17	ECDFs of pipeline duration deviation from overall mean for small repository	68
18	ECDFs of pipeline duration deviation from overall mean for large repository	70
19	ECDFs of pipeline latency	72

List of Tables

1	Key nomenclature differences between GitHub, GitLab, and Bitbucket	35
2	User-centric aspects related to running CI/CD pipelines in GitHub, GitLab, and Bitbucket	65
3	Extended pipeline run metrics for a defined close time interval for GitHub small repository and provider-hosted runner environment	74
4	Calculated expected wait in queue times from the Table 3 - see Section 2.4 for exact formulas	75
5	Calculated expected wait in queue times for provider-hosted pipeline runs meeting criterion defined in Eq. 10 - see Section 2.4 for exact formulas	78
6	Calculated expected wait in queue times for self-hosted pipeline runs meeting criterion defined in Eq. 10 - see Section 2.4 for exact formulas	78